# Text-to-Constraints: Mapping Regulatory Requirements to Executable Data Quality Constraints (Extended Version)

Eduardo H. M. Pena
Federal University of Technology-Paraná
Brazil, Campo Mourão
eduardo@utfpr.br

Junio de Freitas
Federal University of Amazonas
Brazil, Manaus
junio.freitas@icomp.ufam.edu.br

Eduardo Cunha de Almeida
Federal University of Paraná
Brazil, Curitiba
eduardo@ufpr.br

Altigran da Silva
Federal University of Amazonas
Brazil, Manaus
alti@icomp.ufam.edu.br

## ABSTRACT

Many regulatory documents specify requirements that valid data must satisfy, but those specifications are rarely available as data quality constraints. Data profiling, particularly constraint discovery, is a well-studied alternative when constraints are unavailable, but discovered constraints reflect what the data (approximately) satisfies rather than what it is required to satisfy. We introduce the *Text-to-Constraints* problem: given a dataset and related regulatory text, map the requirements stated in that text to executable data quality constraints over the dataset schema. Then, we present the first framework to address this problem, `Norma`, which identifies verifiable requirements in regulatory documents and maps them to denial constraints using a rich schema-grounded predicate vocabulary. We evaluate `Norma` on six datasets with diverse textual and data characteristics, covering constraints ranging from simple checks to complex cross-record rules. `Norma` outperforms baselines from data profiling and Text-to-SQL in constraint classification and mapping on all six datasets, recovering constraints they miss and identifying real regulatory violations in publicly released data, including two large mortgage datasets with up to one million records.

**PVLDB Reference Format:**
Eduardo H. M. Pena, Junio de Freitas, Eduardo Cunha de Almeida, and Altigran da Silva. Text-to-Constraints: Mapping Regulatory Requirements to Executable Data Quality Constraints (Extended Version). PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

**PVLDB Artifact Availability:**
The source code, data, and/or other artifacts have been made available at https://github.com/EduardoPena/norma-text-to-constraints.

## 1 INTRODUCTION

Many data management tasks rely on *data quality constraints*, which define what constitutes valid data. However, such constraints are often missing, incomplete, or outdated in real-world datasets. This gap has motivated a large body of work on *data profiling*, where data constraints are automatically discovered from the data itself [2]. Profiling algorithms extract constraints, such as functional dependencies (FDs) and denial constraints (DCs), by identifying patterns that hold over a given data instance, and have been widely used to support data cleaning and error detection [55].

However, this line of work derives constraints only from the data. Exact discovery returns rules that the instance satisfies, approximate discovery tolerates a fraction of violations [38, 44], and rule-discovery systems help experts rank and refine the candidates [19]. In all cases, the candidates are patterns that hold, exactly or approximately, in the data, rather than rules that the data is *required to satisfy* by some policy, regulation, or legislation. A required rule that is widely violated in the released data is therefore hard to recover from the data alone.

Many datasets are governed by rules specified in regulatory documents, policy manuals, and operational specifications, which define requirements such as mandatory fields, valid values, eligibility criteria, uniqueness conditions, quotas, and ordering rules [7, 12, 53]. Formalizing these rules as constraints is difficult because they depend on combinations of attributes, relationships across tuples, comparison operators, thresholds, and context-dependent conditions, requiring expressive *data quality constraints*. However, since such rules are typically expressed only in natural language, they are rarely translated into executable constraints and used with the data they govern. Consequently, datasets are frequently released without an explicit representation of their own correctness requirements, so violations remain undetected and subsequent analyses may be based on data that does not satisfy their intended specification.

Example 1 (Brazilian public university admission). *Brazil's "Sistema de Seleção Unificada" (SISU) manages the public university placements for millions of students annually. The application data is publicly available,*[1] *with sensitive fields anonymized. The dataset schema includes attributes such as* student_id, course_code, shift, institution_id, year, *and* edition. *The rules that define correct enrollment outcomes, however, are described in separate documents that do not accompany the dataset. For instance, Normative Ruling No. 21/2012*[2] *is a PDF file found on the Brazilian government website that states (Article 19):*[3]

[1] https://dadosabertos.mec.gov.br/sisu
[2] https://sisugestao.mec.gov.br/docs/portaria-2012-21.pdf
[3] Schema elements and quoted statements translated from Portuguese for readability.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

> *"... the student will be ranked in descending order of scores for the seat option they applied to, respecting the available seats at the institution, by campus location, course, and shift, as well as the student's admission category."*

*This passage defines an* ordering constraint*: within each context defined by institution, campus, course, shift, and admission category (and per year/edition), a higher score should not result in a worse rank. The following SQL query checks for violations of this requirement in SISU's application data:*

```
SELECT t.student_id, t'.student_id
  FROM applications t, applications t'
 WHERE t.student_id <> t'.student_id
   -- same context
   AND t.year = t'.year
   AND t.edition = t'.edition
   AND t.institution_id = t'.institution_id
   AND t.campus_id = t'.campus_id
   AND t.course_code = t'.course_code
   AND t.shift = t'.shift
   AND t.application_mode = t'.application_mode
   -- violation: higher score but worse ranking
   AND t.score > t'.score
   AND t.ranking > t'.ranking;
```

*Any nonempty result set for this query indicates a violation of a legal requirement. On the released data, the query returns 76 violating pairs involving 61 applications.*

□

In this paper, we introduce the *Text-to-Constraints* problem: given a dataset and a set of normative requirements derived from related regulatory text, map those requirements to executable data-quality constraints over the dataset schema. This problem differs from traditional constraint discovery in that, instead of inferring constraints from patterns observed in the data instance, which may reflect patterns that hold only incidentally rather than intended rules, it requires reasoning over regulatory semantics, schema structure, and violation logic under a chosen constraint formalism. The *Text-to-Constraints* problem is related to the well-known Text-to-SQL [54] problem as both involve mapping natural-language specifications to structured representations grounded in a database schema. However, Text-to-SQL generates queries to answer specific user questions, whereas Text-to-Constraints maps normative requirements to executable constraints that must hold for all valid dataset instances.

**Challenges.**

Regulatory documents consist of natural-language text, including sentences that carry normative force [16] (i.e., sentences that prescribe what the data is required to satisfy). Thus, the first challenge we address is mapping these natural-language rules that define valid data to executable data-quality constraints. Example 1 illustrates this challenge: Article 19 of the SISU regulation states an ordering requirement in prose, while checking it requires making explicit the relevant context, attributes, tuple comparisons, and violation condition. More generally, a normative passage may combine conditions, qualifiers, cross-record relationships, numeric thresholds, and exceptions, while leaving implicit how its terms correspond to elements of the dataset schema. Mapping such a passage to an executable constraint therefore requires resolving its semantics, identifying the relevant schema elements, and specifying what constitutes a violation.

A second challenge is determining the correct scope of each constraint. Omitting qualifiers results in overly broad constraints, flagging valid data as violations. Adding conditions to the rule results in constraints that are too narrow, missing violations. In Example 1, omitting qualifiers such as year/edition or admission category mis-scopes the check and can flag valid outcomes.

A third challenge is the choice of the target formalism. Any executable check can be written in SQL, but the same rule admits many syntactically different, equivalent queries, so SQL alone offers no canonical form against which mapped rules can be validated, compared, and reused across documents. More restricted languages are canonical and easy to evaluate, but they miss many cross-record rules, including ordering, quotas, and conditional uniqueness. Example 1 requires comparing applicants within the same context, so we need a canonical constraint language that supports such comparisons and can still be compiled into efficient SQL checks on large datasets (Section 2).

Large language models provide a promising way to perform this mapping from text to constraints, as demonstrated in related regulatory compliance tasks [17]. But Text-to-Constraints requires more than identifying relevant passages. The output must be linked to the dataset schema, conform to a constraint formalism, and express violation conditions that can be executed against data. None of these requirements follows automatically from language model outputs. They require explicit mechanisms for schema alignment, predicate grounding, and violation semantics. Other lines of work also bring richer rules or learned components to data quality, but they start from the data. Rule languages such as REEs embed ML predicates in logic rules for data cleaning [6], and LLMs now propose candidate rules from schemas and data samples [8, 22]. Extending DC discovery with richer predicates would still take the data as the only evidence. `Norma` instead takes the rules from the normative text and uses the data only to ground and validate them, through a controlled, schema-derived predicate vocabulary.

**From text to data quality constraints.**

To address these challenges, we propose `Norma`, the first framework for Text-to-Constraints. Given the sentences of a regulatory text and a target dataset, `Norma` optionally filters the sentences with normative force and maps each of them to a formal constraint over the dataset schema. `Norma` uses denial constraints to represent constraints, as they provide a striking balance between expressiveness and control. Unlike Text-to-SQL systems, which generate ad hoc SQL queries in response to user questions, `Norma` derives reusable constraints with well-defined violation semantics. It also differs from traditional data profiling. In data profiling, denial constraints are defined over a narrower set of predicates for tractability. Because `Norma` operates over text instead of searching the data instance for patterns that happen to hold in it, it can use a richer predicate vocabulary. This includes multiple operators, column relationships, null checks, domain checks, and pattern predicates. As a result, `Norma` can capture a broader range of regulatory rules than traditional profiling methods, while still keeping the output space structured and restricted.

Norma supports classification and mapping implementations with different cost and quality trade-offs, from single-prompt variants to a structured multi-step mapper. Our experiments show that this framework recovers rules that state-of-the-art data profiling algorithms and Text-to-SQL systems do not capture. Norma identifies real compliance failures in publicly released data using constraints mapped from the governing text. A key advantage of this setting is that each detected violation can be traced back to the source passage that justifies the constraint.

A summary of our contributions is the following:

- A formalization of the *Text-to-Constraints* problem;
- Norma, the first framework for this problem, with sentence-level classification and mapping implementations with different cost-effectiveness trade-offs, and a compilation of the mapped constraints into SQL violation queries;
- A set of datasets with annotated constraints, and evaluation measures adequate for the Text-to-Constraints problem, including two large real-world datasets from mortgage regulation;
- An extensive empirical study across multiple models, prompting strategies, and baselines from data profiling and Text-to-SQL, showing that Norma recovers critical constraints missed by data-only profiling algorithms.

The remainder of this paper is organized as follows. Section 2 formalizes the Text-to-Constraints problem. Section 3 introduces Norma, whose classifier and mapper components are detailed in Sections 4 and 5. Section 6 presents our experimental evaluation of Norma against different baselines. Section 7 discusses related work, and Section 8 presents our conclusions.

## 2 PRELIMINARIES

**Problem definition.** We are given a relational dataset $\mathcal{D}$ and an associated *normative document*. This textual source could be laws, regulations, policy manuals, or operational specifications. It defines the requirements the data must satisfy, expressed in natural language rather than formal, machine-executable constraints.

We model a normative document as a set $\mathcal{S}$ of *segments*, with each segment $s \in \mathcal{S}$ representing a distinct, self-contained textual unit in the document. The segmentation granularity may vary. A segment can correspond, for example, to a single sentence, multiple sentences, or a larger passage. We consider a document normative if it contains at least one segment that expresses a requirement whose compliance can be assessed based on the data.

A *data quality constraint* is a formal rule that $\mathcal{D}$ must satisfy (we discuss the choice of constraint formalism below). Constraints are evaluated on a target relation $r \in \mathcal{D}$ with schema $R = (A_1, \ldots, A_n)$, where each $A_i$ is an attribute with domain $\mathrm{dom}(A_i)$. If a constraint involves multiple relations, we evaluate it on a derived relation obtained by applying the required joins.

Let $\Phi$ denote the space of constraints over $R$. The *Text-to-Constraints* problem is to produce a set of segment–constraint pairs

$$\Pi = \{(s_1, \varphi_1), \ldots, (s_k, \varphi_k)\} \subseteq \mathcal{S} \times \Phi,$$

where $s_i \in \mathcal{S}$ is a segment and $\varphi_i \in \Phi$ is a constraint whose semantics are grounded in the normative content of $s_i$. Each pair must satisfy two requirements: *executability*, $\varphi_i$ can be evaluated on any instance of $R$, and *traceability*, $s_i$ is identifiable as the textual justification for $\varphi_i$.

In general, one segment may yield multiple constraints, and a constraint may depend on multiple segments. In this work, we consider sentence-level mapping, where each segment is a single sentence and yields at most one constraint, which may consist of several independent DCs whose violations are united (Section 5.4); coarser granularities, in which constraints are derived by jointly reasoning over several segments, are left for future work.

**Denial constraints.** We instantiate $\Phi$ as the space of *denial constraints* (DCs). A DC specifies that a conjunction of predicates must never be simultaneously true. Formally, given a relation instance $r$ over schema $R$, a DC $\varphi$ is a first-order sentence of the form:

$$\varphi : \forall t_1, \ldots, t_m \in r : \neg(p_1 \wedge \cdots \wedge p_\ell),$$

where each $p_i$ is an atomic predicate over tuple variables and constants, and $m \geq 1$ is the number of tuple variables. While DCs may in principle involve an arbitrary number of tuple variables, we focus on DCs with $m \leq 2$. This restriction is common in the literature and adopted by prior work [14, 37, 44], as DCs involving more than two tuple variables are rare in practice and lead to a dramatic increase in both search and semantic complexity.

Let $A, B \in R$ and let $c$ denote a constant in $\mathrm{dom}(A)$. We allow the following predicate forms: (i) comparisons $u\,\theta\,v$ with $\theta \in \{=, \neq, <, \leq, >, \geq\}$ where $u$ and $v$ are tuple attributes (e.g., $t.A$, $t'.A$, $t.B$), constants, or arithmetic expressions over numeric attributes of the same tuple (e.g., $5 \cdot t.B$); (ii) pattern predicates $t.A$ LIKE $p$ and $t.A$ NOT LIKE $p$ where $p$ is an SQL pattern; (iii) set-membership predicates $t.A$ IN $C$ and $t.A$ NOT IN $C$ where $C \subseteq \mathrm{dom}(A)$; and (iv) null checks $t.A$ IS NULL and $t.A$ IS NOT NULL.

A DC $\varphi$ is *violated* if there exist tuples for the quantified variables such that all predicates hold simultaneously.

DCs can be translated directly to SQL. For example, the SQL query in Example 1 is actually equivalent to the following DC:

$$\varphi : \forall t, t' \in \mathit{applications} : \neg\begin{pmatrix} t.year = t'.year \ \wedge\ t.edition = t'.edition \wedge \\ t.institution_id = t'.institution_id \ \wedge\ t.campus_id = t'.campus_id \wedge \\ t.course_code = t'.course_code \ \wedge\ t.shift = t'.shift \wedge \\ t.application_mode = t'.application_mode \wedge \\ t.score > t'.score \ \wedge\ t.ranking > t'.ranking \end{pmatrix}.$$

**Why DCs.** DCs provide a compact formalism for a broad class of integrity constraints, including checks, keys, functional dependencies, order dependencies, conditional variants, and other cross-row conditions. They are also directly checkable on a relation through their violation semantics, making them a natural target for compliance-oriented rule extraction. Compared to SQL, which allows many syntactically different but equivalent formulations of the same rule, DCs offer a more uniform and canonical representation, yielding a more stable target space for extraction and evaluation. Finally, with the standard restriction to at most two tuple variables, the space of candidate constraints remains manageable in practice.

## 3 SYSTEM OVERVIEW

This section presents Norma, our proposed framework for the Text-to-Constraints problem.

Intuitively, one can address this problem by using an LLM with a general prompt that includes the entire document, the dataset schema, and instructions for extracting constraints. However, document-level prompting faces structural challenges, as relevant statements may be scattered across the text and the model must jointly identify relevant parts of the text, align them with the schema, and construct valid constraints. Such a prompt can still produce useful constraints, but it does not link, by construction, each constraint to the text that justifies it. Recovering this link requires an attribution step, for instance an iterative pipeline that first summarizes related sentences and then maps each summary.

We therefore decompose the problem into a pipeline that operates at the sentence level, processing each sentence individually. This design provides fine-grained traceability without an attribution step, since it links each constraint to the specific sentence that justifies it, and allows each component of the pipeline to be tuned independently.

`Norma` takes as input a normative document, segmented into sentences, and a relational dataset and outputs sentence–constraint pairs. Figure 1 outlines the pipeline.

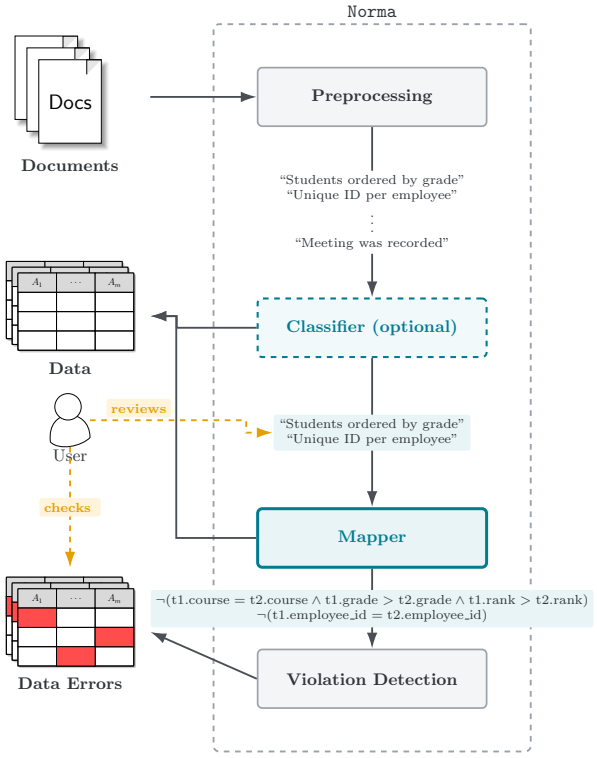



**Figure 1: Overview of the `Norma` pipeline. Blue components represent the novel contributions presented in this paper.**

**Preprocessing.** We start from documents (e.g., PDF files) and produce a normalized text representation. This step can be noisy (e.g., leaked headers, split tokens, and flattened lists). Since our goal is Text-to-Constraints rather than document conversion, we treat preprocessing as input preparation and do not focus on it in this work. We run an automated pipeline covering text extraction, dehyphenation, whitespace normalization, and sentence segmentation, followed by minimal manual cleanup.

**Constraint classifier.** Given the sentences from preprocessing, a lightweight classifier can optionally identify those that likely express checkable constraints, filtering out content that is clearly unrelated to data quality constraints. Since constraints missed at this stage cannot be recovered downstream, we prioritize recall at this stage. False positives can still be ruled out by the mapper (which we define next). The classifier only labels the given sentences and never generates text, so it preserves the link between each sentence and its constraint. When no classifier is used, all sentences are passed directly to the mapper. A user can optionally be involved at this stage, reviewing and correcting the set of classified sentences before proceeding to mapping. We describe the classifier details in Section 4.

**Constraint mapper.** The mapper is the central and most complex component of `Norma`. It takes each sentence, optionally pre-filtered by the classifier, and produces a denial constraint grounded in the dataset schema. The mapper must interpret natural language, align it with schema elements, and synthesize a constraint with correct violation semantics. Outputs that fail syntactic validation are rejected, and sentences that yield no valid constraint are excluded from the final output. We describe the different mapper implementations in detail in Section 5.

**Violation detection.** Each constraint is validated by compiling it into a SQL violation query and executing it on the target relation (Section 5.4), and it is linked back to the sentence that produced it. `Norma` returns the violating tuples to the user for further inspection. DC violation detection is a well-studied problem, with specialized systems available for large-scale consistency checks [28, 35, 45]. We compile DCs into SQL because the queries run on any engine that holds the data and also serve the data-based checks of the mapper; specialized engines can replace this step for larger workloads.

## 4 CONSTRAINT CLASSIFICATION

Given the segment set $\mathcal{S}$ extracted from a regulatory document, the constraint classifier aims at identifying the subset $\mathcal{S}_c \subseteq \mathcal{S}$ of segments that state data quality constraints. A segment is a positive example if it expresses a condition that the data is expected to satisfy and that can be validated against a dataset. Many segments only define terms, describe procedures, or provide context. Constraint-related segments appear less often, and many segments that sound like rules are still too vague to test with data.

For ease of exposition, we treat each segment as a single sentence from now on, unless stated otherwise.

Example 2 (Restaurant inspections). *The NYC Department of Health publishes restaurant inspection results, including scores and letter grades,*[4] *alongside a document describing the grading and inspection regulations.*[5] *Table 1 shows sample sentences from these regulations. The first two sentences state data quality rules that can be validated when the dataset contains the needed information. The first*

[4]Dataset available at https://data.cityofnewyork.us/Health/DOHMH-New-York-City-Restaurant-Inspection-Results/43nn-pn8j. See Section 6 for details.
[5]Available at https://www.nyc.gov/site/doh/services/restaurant-grades.page.

*connects score ranges to letter grades and can be checked by comparing the score and grade fields. The second constraint defines inspection frequency per establishment and can be checked when establishment identifiers and inspection dates are available. The last two are not data quality rules: one is too underspecified to test because it does not define which inspections yield grades, and the other describes user permissions rather than a condition on the data.* □

| *Sentence* | *Constraint?* |
|---|---|
| *A score of 0–13 results in a grade of A.* | *Yes* |
| *Every food service establishment is scheduled for at least one inspection per year.* | *Yes* |
| *Only certain inspections result in a grade.* | *No* |
| *Consumers may check an establishment's inspection history.* | *No* |

**Table 1: Constraint vs. non-constraint sample sentences from NYC restaurant inspection regulations.**

Norma implements the classifier via *LLM-based prompting*, which can incorporate schema and data samples at inference time and generalizes across domains without requiring labeled training data.

## Constraint Classification by Example

We ask a large language model to label each sentence as either stating a data quality rule or not. Figure 2 shows how the prompt is generally structured. Our design is inspired by few-shot in-context learning [11], where a model is guided by a small number of task examples provided in the input context. In our case, these examples are generic and independent of the target domain, serving to guide the model's structured reasoning over candidate constraints. The model returns one label per numbered sentence through a JSON schema. Malformed outputs are retried, then the sentences of the batch are classified one at a time, and a sentence that still has no valid label is marked as non-constraint. Section 6 compares closed- and open-weight models using these prompt versions.

**Prompt structure.** Following standard practice in prompt-based text classification [57], we include a task definition, category descriptors, and examples. These elements are essential because constraint classification presents two main challenges. The first challenge is that the positive class is rare and spans many syntactic forms. To reduce false negatives, we enumerate rule types that the model should treat as constraints. Otherwise, it may miss cases such as functional dependencies that might use specific language. The second challenge is that non-constraint sentences can be easily mistaken for constraints. Definitions, procedures, and contextual statements often use similar wording, mention data-related concepts, or contain keywords that suggest a requirement. To reduce this confusion, we explicitly add notes on exclusions and edge cases.

**Examples.** Our approach follows a few-shot prompting strategy [11], using generic templates that remain fixed across datasets, rather than domain-specific examples. As a result, the prompt remains portable without requiring labeled data.

**Table Context.** We evaluated multiple prompt variants. Some ask the classifier to decide from the sentence alone whether it expresses a checkable constraint. Others also provide table context, including table and column information and sample tuples. Without access to the schema, the classifier may produce false positives for sentences that cannot be checked against the available fields. Schema context helps reject such cases and, therefore, mainly improves precision. However, it also increases prompt length and can make predictions sensitive to the particular sample tuples provided.

| | |
|---|---|
| **Task Definition** | For each sentence, decide whether it states a DATA QUALITY CONSTRAINT that can be validated from the dataset when the required fields are available. |
| **Rule Types** | Required/non-null field; Allowed or forbidden values; Numeric bounds; Conditional requirements; Cross-row consistency; Determinacy; Temporal constraints; … |
| **Exclusions** | Headings, titles, citations; Definitions or descriptions; Procedures or human decisions; Statements not testable from the dataset; … |
| **Notes** | Modal verbs ("may", "should") do not disqualify a sentence; what matters is whether it describes a data condition or verifiable property; … |
| **Examples** *(optional)* | YES:<br>"Field X is required."<br>"Two rows with the same Key must not disagree on Field H."<br>…<br>NO:<br>"Field X represents the total amount."<br>… |
| **Context** *(optional)* | Table Schema: [Column Names / Types]<br>Table Sample Rows: [Example Tuples] |
| **Input** | Sentences:<br>[Numbered List of Sentences]<br>Return one YES/NO label per sentence number |

**Figure 2: Constraint classification prompt structure (simplified). Colors indicate: ■ positive label clues, ■ negative label clues, ■ data context, and ■ instructions.**

**Batch processing.** To reduce API overhead, Norma classifies sentences in batches, concatenating multiple sentences into a single prompt using numeric indices and parsing one label per index from the response. We use moderate batch sizes (up to 10 sentences per prompt). Malformed responses trigger a retry.

## 5 CONSTRAINT MAPPING

Let $\mathcal{S}$ denote the sentences passed to the mapper, either all preprocessed sentences or the subset retained by the classifier. For each $s_i \in \mathcal{S}$, the mapper returns a denial constraint (DC) $\varphi_i = \neg(p_1 \wedge \cdots \wedge p_\ell)$ over the dataset schema, or no output if $s_i$ does not express a constraint or cannot be linked to the schema.

Recall from Section 2 that DC predicates can take several forms, including comparisons between an attribute and a constant (e.g., `t.A = c`), between two attributes of the same tuple (e.g., `t.A > t.B`), or between attributes across tuples (e.g., `t1.A = t2.A`), as well as null checks and membership tests over permitted values. The mapping task is therefore to interpret a sentence, identify the relevant schema elements, select the predicates on those elements that capture the constraint intuition, and produce an executable *violation condition*. In this sense, it is related to Text-to-SQL [59], as both tasks aim to connect a natural language segment to schema-aware logical expressions. One key difference is that Text-to-SQL typically retrieves satisfying records, whereas our target is a conjunction of predicates that identify a constraint violation.

Example 3 (Mapping to DC predicates). *Consider the sentence "A score of 0–13 results in a grade of A" from the NYC restaurant inspection regulations. This sentence states that scores in the range 0–13 must be associated with grade A. The corresponding violation condition is:*

$$\texttt{t.score} \geq 0 \land \texttt{t.score} \leq 13 \land \texttt{t.grade} \neq \text{`A'}$$

*Any tuple satisfying all three predicates violates the rule.* □

Mapping regulatory text to DCs is challenging because the same rule may be expressed in many different ways, often with implicit references to schema elements, literals, or tuple relationships. We therefore rely on LLM-based mappers, motivated by their strong performance on related semantic parsing tasks such as Text-to-SQL [54]. However, unrestricted LLM generation is unreliable in our setting: LLMs are known to hallucinate column names and use unsupported operators [18, 21, 54]. We also observed that LLMs tend to output predicates describing valid states rather than violations, so we introduce hints toward violations in the prompt.

To make mapping tractable and reliable, our design follows two principles. First, we derive a *predicate space* from the dataset schema and use it as a controlled vocabulary. The mapper must either select predicates from this space or instantiate typed placeholders within predefined predicate templates. This reduces the search space and prevents ill-formed outputs. Second, the mapper is biased toward *precision*. Incorrect mapping can generate false violations and corrupt downstream checks, whereas an unmapped sentence can still be marked for manual inspection.

We implement two mapper variations that share the same predicate space but differ in how they use it. The *single-prompt mapper* (Section 5.2) performs the full mapping in a single LLM call, while the *multi-step mapper* (Section 5.3) decomposes the task into focused LLM calls interleaved with deterministic checks. Both produce DCs that are compiled into SQL violation queries (Section 5.4).

## 5.1 Predicate Space

Traditional DC discovery operates over a *predicate space*, that is, the set of predicates allowed to appear in candidate constraints [14, 44]. Because discovery methods enumerate and validate combinations of predicates, this space is kept small, which limits expressiveness [14].

Our setting requires a richer predicate space. Regulatory text frequently refers to enumerations, ranges, string formats, and arithmetic conditions. Comparisons with constants and null checks are expressible as DCs, but discovery spaces rarely include set membership, patterns, or arithmetic, because each extension multiplies the candidates to enumerate. We therefore extend the predicate space with typed predicate templates that can be instantiated with literals extracted from text. Importantly, Norma does not enumerate candidate DCs over this larger space. Instead, it uses the space as a controlled vocabulary for mapping, which avoids candidate explosion while preserving syntactic validity.

**Predicate templates.** We derive a finite set of predicates and predicate templates from the schema, while the data instance is used to support specific instantiations and validation steps. We consider the following categories:

(1) **Cross-tuple predicates** compare attributes across two tuples (e.g., `t1.A = t2.A`, `t1.A > t2.B`). We allow $\{=, \neq\}$ for type-compatible attributes and additionally $\{<, \leq, >, \geq\}$ for numeric attributes.
(2) **Same-tuple predicates** compare two attributes within one tuple (e.g., `t1.A ≤ t1.B`), capturing intra-tuple rules.
(3) **Null predicates** (`t1.A IS NULL`, `t1.A IS NOT NULL`) capture requiredness and conditional missingness constraints.
(4) **Attribute–constant predicates** compare an attribute to a constant $c$ (e.g., `t1.A = c`, `t1.A ≥ c`).
(5) **Pattern predicates** encode string-format and substring conditions (e.g., `t1.A LIKE p`).
(6) **Set predicates** encode membership in small enumerations (e.g., `t1.A IN C`).
(7) **Arithmetic predicates** compare an arithmetic expression over numeric attributes of the same tuple with an attribute or a constant (e.g., `t1.A ≥ c * t1.B`), when the schema enables arithmetic for those attributes.

For categories 4–7, the mapper instantiates the placeholders $c$, $p$, or $C$ using literals extracted from the sentence. **Coverage and limits.** Across our six datasets, 487 of the 490 predicates of the gold DCs belong to this space. The three exceptions, on HMDA Checks, compare an integer-encoded date with an arithmetic expression over the reporting year. The categories exclude aggregation and arithmetic over dates or sequences of records, so rules over counts or inspection histories require derived attributes, such as the days until the next inspection in Restaurant (Section 6.1).

**Predicate restrictions.** To avoid semantically uninformative or implausible predicates, we adopt a filtering principle from DC discovery [14]. Order predicates are generated only for numeric attributes. Cross-tuple and same-tuple predicates are generated only for type-compatible pairs and are further pruned using observed value overlap, helping eliminate comparisons between unrelated columns.

**Schema-driven, on-demand predicate space.** Materializing the full space does not scale to wide schemas: with arithmetic and code-set templates, the space of a 135-column table reaches hundreds of thousands of templates, most of which are irrelevant to any given sentence. Norma therefore stores only a *schema description* and generates predicates on demand. For each column, the description records its type, the comparison and arithmetic operators it admits, its domain values when it has at most a few distinct values, and a short natural-language description. Descriptions are written once, offline, by an LLM from a deterministic profile of the data; when an official data dictionary is available, its definitions are merged into the description of the corresponding columns (we do this for HMDAZ, using the FFIEC field definitions). At mapping time, once the relevant columns of a sentence are selected, the predicates of the categories above are generated only for those columns, under the same restrictions. For a fixed instance and fixed options, the generated set equals the full space restricted to the selected columns, so the controlled vocabulary is unchanged; only its materialization is deferred.

## 5.2 Single-Prompt Mapper

The single-prompt mapper performs the full mapping in a single LLM call. We design the prompt around three elements: the schema and predicate space as a controlled vocabulary, a small set

of examples covering the main constraint patterns, and predicate specifications to guide the model in understanding and mapping violation conditions. Given an input sentence, the model returns either a list of rules over the retrieved columns and their predicates or `null`. Figure 3 summarizes the prompt structure. The prompt is restricted to the columns retrieved for the sentences of each batch (three sentences per call): columns are ranked by the embedding similarity between the sentence and the column name and description, and the prompt carries the description and domain values of each retrieved column together with the predicates generated for them. The model must also state, for each sentence, the condition under which a tuple violates the rule (`violated_when`) before writing the DC, which makes the violation form explicit.

| | |
|---|---|
| **Definition** | A denial constraint (DC) is a conjunction of predicates that *cannot* all hold simultaneously in a dataset. DC predicates must encode *violations* of data quality rules, not valid states. |
| **Violation Form** | "must be X" ⇒ `!= X`; "must not be X" ⇒ `= X`; |
| **Cross-Tuple** | Signals such as "same …", "repeated", or "consecutive" introduce `t1`, `t2`, and a join predicate on the shared key. |
| **Context** | Table schema + top-$k$ frequent values per column.<br>List of possible predicates (predicate space). |
| **Examples** | "Value must be at least 100" ⇒ `["t1.value < 100"]`<br>"Same ID must share status" ⇒ `["t1.id = t2.id", "t1.status != t2.status"]`<br>"The system performs well" ⇒ `null` |
| **Output** | Array of length $n$, one entry per input sentence; `null` for non-constraints. Predicates must come from the predicate space. |

**Figure 3: Prompt structure for the single-prompt mapper. Examples are representative, as the full prompt covers threshold, set-membership, field-comparison, null, cross-tuple, and non-constraint cases. The colors represent: ■ Data context; ■ constraint example; ■ non-constraint; ■ instructions.**

**Violation form.** Without explicit guidance, LLMs generally tend to output predicates describing *satisfying* tuples rather than *violating* ones. Since regulatory sentences typically describe what must hold, the mapper must find predicates expressing a violation condition—as in Example 3, where the correct output is `grade ≠ 'A'`, not `grade = 'A'`. We make this criterion explicit in the prompt through a small set of examples.

**Cross-tuple constraints.** Some rules address relationships between records rather than conditions within a single record. In these cases, the mapper must introduce two tuple aliases and a join predicate on the shared identifier. Including cues such as "for the same" in the prompt already suffices for the model to generalize reliably to the variety of phrasings in which such predicates appear. The target remains a DC with at most two tuple variables and universal violation semantics: a single violating pair suffices. Rules that must hold only for most pairs, or that compare distances between values, such as differential dependencies, would require counting or tolerance semantics and are outside our scope.

**Literal grounding.** Many constraints require grounding numeric thresholds, strings, or small enumerations. A recurring failure is to produce literals in the wrong format or casing. To mitigate this, we include in the prompt the description and the domain values of each retrieved column, which anchors literals to the dataset representation (e.g., a label such as `'Secured by a first lien'` rather than a paraphrase).

The mapper returns for each sentence a list of independent rules, whose violations are united (Section 5.4). If the sentence is not mappable, it returns `null`. We parse the output and validate every predicate against the schema: it must reference existing columns, use operators admitted by their types, compare them with constants of the right type, and have no uninstantiated placeholders. Membership in the generated predicate space is not enforced for this mapper; we report it instead (out-of-space predicates). Malformed outputs are retried a fixed number of times, and invalid rules are returned once to the model with the error. If all attempts fail, the sentence produces no constraint. To prevent incorrect mappings from affecting downstream checks, we intentionally keep this stage conservative.

**Batch processing.** As in the classifier (Section 4), we support batch processing by concatenating multiple indexed sentences into one prompt and returning one output per index.

## 5.3 Multi-Step Mapper

The single-prompt mapper needs to parse several subtasks in one call: schema linking, predicate selection, literal linking, and violation form. In this setting, errors in early steps might sometimes propagate. For example, a wrong column choice may break predicate selection, a hallucinated literal yields a syntactically valid but semantically meaningless constraint, and a non-constraint sentence triggers an unnecessary mapping attempt. Recent work in Text-to-SQL shows that decomposing semantic parsing into multiple stages, especially separating schema linking from later query building decisions, often improves accuracy [18, 21, 51, 54].

We propose a multi-step mapper that breaks down mapping into several LLM calls, with deterministic checks in between. The process uses four LLM calls, where each call receives the output from the previous step. Sentences that cannot be mapped are removed before constraint construction. After construction, data-driven checks eliminate constraints that have incorrect literals or implausible violation sets. Each stage isolates one decision that the single-prompt mapper must make jointly (columns, DC structure, whether the sentence is a rule, and literals), so the stages trade additional calls for smaller decisions that can each be validated. In our experiments, this decomposition keeps the mapper usable with backbones whose single-prompt outputs fail, and the rule filter rejects most non-constraint sentences without a construction call (Section 6). A finer ablation that removes or reorders individual stages is left for future work.

Figure 4 illustrates the control flow of our multi-step mapper and Figure 5 provides a summary of its prompts in each step.

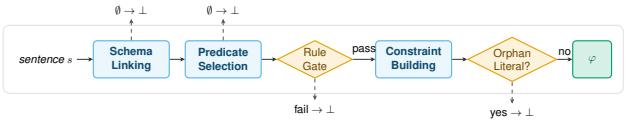



**Figure 4: Multi-step mapper control flow.**

| Step | Context | Task |
|---|---|---|
| **Schema** | Full schema, column profiles, predicate space | Identify all columns relevant to $s$: direct mentions, synonyms, and derived concepts. Abort if no concept maps to any column with defined predicates. |
| **Predicates** | ↪ Columns from Schema step<br>Predicate space restricted to those columns<br>Column profiles | Select predicates and predicate templates from the restricted predicate space and identify the constraint pattern for $s$ (value mapping, temporal bound, functional dependency, required field, or conditional rule). Abort if no predicate or predicate template fits. |
| **Rule filter** | ↪ Columns and predicates from Predicates step | Decide whether $s$ expresses a specific, data quality rule verifiable from column values alone.<br>Reject: process descriptions, permissions, vague quantifiers, physical conditions.<br>Accept: value mappings, temporal bounds, functional dependencies, required fields, conditional rules. |
| **Construction** | ↪ Predicates and predicate templates from Predicates step<br>Column profiles | Substitute predicate placeholders with literals from $s$ and return the constraint in violation form: a conjunction of predicates that must not hold simultaneously. |

**Figure 5: Prompt structure of the multi-step mapper. Context narrows progressively. The rule filter runs after predicate selection but before construction, avoiding unnecessary LLM calls on non-rules. ■ input context (variable per step); ■ accept signal; ■ abort / reject signal; ■ fixed instruction.**

**Schema linking.** This step identifies the columns relevant to the sentence, including implicit references and derived concepts. If no column with defined predicates can be aligned to any concept in the sentence, we reject the sentence for mapping.

**Predicate selection.** The mapper uses only columns retained by schema linking. Given the column names, profiles, and restricted predicate spaces, it selects predicates and predicate templates (requiring literals) that together form the DC pattern. This separation aligns with prior results in text-to-SQL, where having a dedicated structural parsing step has been shown to improve accuracy [30].

**Rule filter.** This step determines whether the sentence can be mapped to a DC under the schema-grounded interpretation obtained so far. Unlike the classifier in Section 4, the rule filter has access to the candidate columns and predicates selected for the sentence. This improves the semantic reliability of the decision, since many sentences that appear rule-like in isolation support no executable constraint once grounded in the schema.

**Constraint building.** This step instantiates the selected predicate templates with the literals needed to form the final DC. The model resolves value-level details such as numeric thresholds, categorical constants, intervals, string patterns, and short enumerations, and aligns them with the dataset representation using column profiles and sample values. The output is a set of predicates in violation form, which are then assembled into the final DC. Framing this stage as grounded instantiation rather than open-ended generation substantially narrows the construction problem and makes unsupported predicates and malformed expressions less likely [33, 48, 52].

**Post-construction checks.** After constructing a candidate DC, the multi-step mapper applies a set of validation checks to ensure that the generated constraints are both executable and semantically plausible. First, we verify the structural validity of the DC: each predicate must belong to the predicate space, reference valid schema columns, use type-compatible operators, and contain no uninstantiated placeholders. Next, we apply a data-based grounding check for unsupported literals. For each predicate with literals involving equality, string inequality, `LIKE`, or set membership, we issue an existence query against the dataset. If no tuple can satisfy the predicate, we refer to it as an *orphan predicate*, i.e., a predicate that is syntactically valid but unsupported by the data instance. Any constraint containing an orphan predicate is discarded, although it may be retained for documentation purposes. Finally, we execute the DC and discard it when more than 90% of the tuples take part in a violation: a DC that flags almost every tuple most likely encodes the valid state rather than its violation. The single-prompt mapper applies only the schema validation described in Section 5.2. Numeric bounds and `null` predicates are excluded from this check because their validity is structural rather than value-based. If parsing fails, the mapper retries a small fixed number of times; if all attempts fail, no constraint is returned.

## 5.4 From Denial Constraints to SQL

Every DC produced by `Norma` is compiled into a SQL *violation query*, which is used for the data-based checks of the mappers, for violation detection, and for evaluation. The compilation is deterministic, but it has to settle several questions that the DC formalism leaves open when it meets real data.

**Rule structure.** For each sentence, the mapper returns a list of independent rules. Each rule is a tree of `AND`/`OR` nodes over the predicate forms of Section 2. A rule whose body contains disjunctions is equivalent to a set of DCs, one per disjunct of its disjunctive normal form, so it remains a denial constraint. Independent rules of the same sentence are compiled separately, and their violations are united.

**Compilation.** A rule with one tuple variable becomes a selection over the relation that returns the identifiers of the violating tuples. A rule with two tuple variables becomes a self-join with the condition $t_1.tid \neq t_2.tid$. Membership and pattern predicates map to `IN` and `LIKE`, division is guarded against zero divisors, and literals are written in a canonical form (e.g., `1` for `1.0`). The self-join condition must not be replaced by an ordering such as $t_1.tid < t_2.tid$, a common way to avoid reporting a pair twice. That shortcut is only correct for symmetric DCs, such as functional dependencies. For an asymmetric DC, such as the ordering rule of Example 1, it misses every violating pair whose witnesses appear in the opposite order: on SISU, the ordering rule flags 61 applications with $\neq$ but only 9 with $<$. Since violations are reported at the tuple level, enumerating both orders does not inflate the counts.

**Types and missing values.** Published data rarely matches its declared schema. Numeric fields contain codes such as `NA` or `Exempt`. We therefore read the data as text, type each column with the schema, and compare numeric attributes through safe casts (`TRY_CAST`), which turn malformed values into `NULL` rather than aborting the query. Under SQL's three-valued logic, a predicate over `NULL` is

unknown, so the tuple is not reported. However, it means that requiredness must be stated explicitly with `IS NULL`, and that a disequality such as `loan_type != 1` silently skips tuples without a loan type. The mapper prompts illustrate requiredness with `IS NULL` predicates and ask for missing values to be written that way.

**Execution.** Queries run on DuckDB over the published files. Each query therefore runs under a memory budget, a time limit, and a cap on the result size. A query that exceeds them is reported as an execution error, never as a constraint without violations.

**Bridging text and data.** Most of the difficulty lies before compilation, in the inferences that the mapper must make so that the compiled query checks what the text means. Table 2 shows real constraints from our datasets. On Tax, the comparative "lower salary cannot have higher tax rate" becomes an asymmetric pair DC. On SISU, "selected in only one of their options" requires the mapper to identify the student by the national ID rather than by the exam registration number, a choice that the text leaves implicit. On HMDA Checks, monetary amounts are recorded in thousands, so "$1,000 ($1 million)" becomes `loan_amount >= 1000`, and "five times the income" becomes an arithmetic predicate. On HMDAZ, the regulation compares the annual percentage rate with the average prime offer rate, a difference that the register stores as `rate_spread`. The trigger also combines coverage conditions, three alternative thresholds, and label columns whose values (e.g., `'Secured by a first lien'`) must match the data verbatim. Each of these decisions changes the violating tuples, which is why violation detection is reported next to structural mapping in our evaluation.

## 5.5 Methodological Contributions

Table 3 delineates which components of Norma build on Text-to-SQL practice and which are specific to mapping normative text to constraints. Prompts implement individual decisions, but replacing a prompt changes neither the vocabulary, the violation form, nor the checks that every constraint must pass.

# 6 EXPERIMENTAL EVALUATION

We implemented a prototype of Norma and evaluated it on several datasets prepared for the *Text-to-Constraints* setting. Our evaluation is organized around the following questions.

- **RQ1: Constraint-sentence classification.** How accurately can Norma identify which sentences in a normative document express data constraints?
- **RQ2: Mapping quality.** Given constraint-relevant text, how accurately can Norma map it to executable denial constraints and how well do these constraints capture the corresponding data violations?
- **RQ3: Impact of implementation choices.** How do different classifier and mapper implementations, LLM backends, and prompt variants affect mapping quality and robustness?
- **RQ4: Comparison with baselines.** How does Norma compare with data profiling and Text-to-SQL baselines on constraint mapping and violation detection?

## 6.1 Datasets

We evaluate Norma on six datasets: four based on real-world regulatory documents paired with corresponding tabular datasets, and two derived from benchmark datasets commonly used in data quality and profiling research. A summary of these datasets is provided in Table 4, including the number of rows and columns, the number of sentences in the source text, the number of annotated constraints, and the number of tuples violating at least one constraint. For every dataset, the authors labeled each sentence and wrote its gold DC in violation form over the released schema; sentences not checkable on the released attributes are non-constraints. No sentence occurs twice, and near-identical sentences (similarity $\geq 0.9$) are templated statements that differ in one attribute or literal (13 pairs in SISU, 32 in HMDA Checks, at most two elsewhere), never sharing a gold DC.

**SISU.** As discussed in Section 1, the SISU dataset contains admission records from Brazil's centralized university selection system. We manually annotated three sections of *Portaria Normativa No. 21/2012* covering student registration, candidate classification and selection, and waitlist procedures. The resulting constraints span 11 nullability checks, a functional dependency, an order dependency (Example 1), a conditional functional dependency (a student is selected in at most one option), and three conditional DCs over one or two tuples (e.g., a student selected in the first option cannot be selected in the second). Many have violations in the released data, suggesting inconsistencies between the published records and the regulatory requirements expressed in the text.

**Restaurant.** As introduced in Example 2, this dataset contains restaurant inspection records published by the Department of Health and Mental Hygiene of New York. We annotated the NYC food service operator inspection guide, focusing on the grading and closure sections, which present constraints on score-to-grade mappings, inspection frequency, grade posting timing, and closure conditions. Most have natural violations and can be grounded directly to existing fields. For the one constraints that reasons over inspection sequences, we added one derived column that tracks the days until the next inspection. We leave to future work how Norma can guide or automate such dataset transformations more systematically.

**Tax and Hospital.** Both datasets have been widely used in data profiling and denial-constraint research since the seminal work of Chu et al. [14]. For Tax, we extracted the constraint descriptions from the original paper [14]. For Hospital, we used the data and constraints from a well-known cleaning system [50]. Because their constraints are available in formal form, we first generated natural-language descriptions with an LLM. For both datasets, we then prompted an LLM to generate additional non-constraint sentences around each rule, mimicking a regulatory document that interleaves binding rules with other prose. The prompt receives the schema and is instructed to use schema terminology, making sentence classification more challenging. To avoid circularity between data generation and evaluation, we used Claude 4.5 Sonnet, a model not evaluated in any Norma configuration. These two datasets nevertheless form a controlled setting: their constraints come from prior papers and their sentences from an LLM that may share priors with the evaluated backbones, which likely makes them easier than real

| Dataset | Sentence (abridged) | Violation condition in the compiled query (`WHERE` clause, casts omitted) | Viol. |
|---|---|---|---|
| Restaurant | A score of 0–13 results in a grade of A. | `t1.score >= 0 AND t1.score <= 13 AND t1.grade != 'A'` | 402 |
| Tax | Within the same state, a person with lower salary cannot have higher tax rate. | `t1.tid <> t2.tid AND t1.state = t2.state AND t1.salary < t2.salary AND t1.rate > t2.rate` | 363 |
| SISU | The student will be selected in only one of their options.† | `t1.tid <> t2.tid AND t1.cpf = t2.cpf AND t1.edicao = t2.edicao AND t1.ano = t2.ano AND t1.etapa = t2.etapa AND t1.opcao != t2.opcao AND t1.aprovado = 'S' AND t2.aprovado = 'S'` | 149 |
| HMDA Checks | If loan amount and income are > 0 and the loan amount is ≥ $1,000 ($1 million), then loan amount should be < 5 times the income. | `t1.loan_amount > 0 AND t1.applicant_income > 0 AND t1.loan_amount >= 1000 AND t1.loan_amount >= (5 * t1.applicant_income)` | 600 |
| HMDA Checks | If purchaser type = 1 or 3, then loan type should = 1. | `t1.purchaser_type IN (1, 3) AND t1.loan_type != 1` | 441 |
| HMDAZ | A high-cost mortgage is a consumer credit transaction secured by the consumer's principal dwelling in which the APR exceeds the APOR by more than (A) 6.5 points for a first lien, (B) 8.5 points for a first lien on personal property under $50,000, or (C) 8.5 points for a subordinate lien. | `t1.business_or_commercial_purpose = 2 AND t1.occupancy_type_name = 'Principal residence' AND t1.reverse_mortgage_name = 'Not a reverse mortgage' AND t1.hoepa_status_name = 'Not a HOEPA loan' AND ((t1.lien_status_name = 'Secured by a first lien' AND (t1.manufactured_home_secured_property_type != 2 OR t1.loan_amount >= 50000) AND t1.rate_spread > 6.5) OR (t1.lien_status_name = 'Secured by a first lien' AND t1.manufactured_home_secured_property_type = 2 AND t1.loan_amount < 50000 AND t1.rate_spread > 8.5) OR (t1.lien_status_name = 'Secured by a subordinate lien' AND t1.rate_spread > 8.5))` | 4 |

**Table 2: Gold constraints of our datasets compiled into violation queries, with the number of violating tuples (HMDA: first 1M rows). Each query selects the identifiers of the tuples bound to `t1` (and `t2`) that satisfy the condition; numeric attributes are compared through `TRY_CAST`. †Translated from Portuguese.**

| Component | Origin | What is specific to `Norma` |
|---|---|---|
| Schema linking | Adapted [30, 48] | Over a schema description with column descriptions and domain values (Section 5.1). |
| Staged decomposition | Adapted [48] | A rule filter decides whether the text is a checkable rule at all; every Text-to-SQL question has an answer. |
| Predicate space | New | Typed templates derived from the schema and generated on demand for the selected columns. |
| Violation form | New | The output is the condition that must never hold (a DC), a canonical target instead of a query. |
| Data-grounded checks | New | Orphan literals and the share of violating tuples are checked on the instance. |
| Compilation | New | Pair, type, and `NULL` semantics of the violation queries (Section 5.4). |
| Evaluation | New | Equivalence-aware mapping, execution equivalence, and per-rule metrics (Section 6.3). |

**Table 3: Components of `Norma` adapted from Text-to-SQL and components specific to Text-to-Constraints.**

| Dataset | #Rows | #Cols | Sent. | Const. | Viol. |
|---|---|---|---|---|---|
| Hospital | 1k | 17 | 109 | 15 | 834 |
| Restaurant | 128k | 21 | 33 | 7 | 52,311 |
| SISU | 102k | 56 | 42 | 17 | 533 |
| Tax | 100k | 15 | 126 | 6 | 468 |
| HMDAZ† | 1M | 135 | 17 | 9 | 209 |
| HMDA Checks† | 1M | 81 | 158 | 92 | 986,491 |

**Table 4: Dataset statistics. †First 1M rows of the public file (15.1M and 4.8M rows); *Const.* counts sentences mapped to a non-null gold DC.**

regulatory text. SISU, Restaurant, and the HMDA datasets, which pair real regulatory text with real data, are the more reliable signal.

**HMDAZ and HMDA Checks.** Both datasets use public loan-level records released under the Home Mortgage Disclosure Act (HMDA); we use the first 1M rows of each file. *HMDAZ* pairs the 2018 loan application register (135 columns, including the code and label versions of each categorical field) with Sections 1026.32, 1026.35, and 1026.43 of Regulation Z, which define high-cost (HOEPA) and higher-priced mortgages, the limitations that apply to them, and the ability-to-repay requirements. Sentences are verbatim from the regulation, and gold DCs keep the coverage conditions of each trigger. Scope-only text, or text whose outcome is not observable in the data, is annotated with a `null` constraint. *HMDA Checks* pairs the 2014 register (81 columns) with the 2014 HMDA edit specification published by the FFIEC: one entry per edit, with the edit test verbatim; edits that cannot be checked on a single record (aggregate, transmittal-sheet, or external-reference edits) are `null`. These datasets stress three aspects that the other datasets lack: wide schemas whose relevant column is often named after a code rather than the concept in the text, rules that compare columns with each other or through arithmetic, and a highly concentrated violation distribution (Section 6.3).

## 6.2 Baselines

Since no prior system addresses the Text-to-Constraints task directly, we compare `Norma` against the two closest paradigms: DC discovery from data alone and Text-to-SQL.

**Data profiling.** We use the DC discovery algorithms from [46], running both exact and approximate denial-constraint discovery. Exact discovery returns only constraints that hold on the data with no violations. As a result, it is not suitable for our violation-based evaluation, which aims to recover rules that may be violated in the released dataset. We therefore focus on approximate DC discovery as the main profiling baseline. Approximate DCs allow a small fraction of violating tuple pairs, controlled by a threshold $\varepsilon$, and can therefore recover rules even when the data is not fully clean. Since

FDs and order dependencies with at most two tuples are special cases of DCs, approximate DC discovery over the same attributes subsumes their discovery, and we do not run separate FD or OD algorithms. Conditional variants require predicates with constants, which DC discovery supports only for a restricted set of values; a comparison with dedicated CFD discovery is left for future work.

Because discovery can return many DCs, we rank them using the standard criterion from prior work [14], which combines coverage and succinctness, with succinctness measured as the inverse of the number of symbols in the DC. We run exact discovery and $\varepsilon \in \{0.001, 0.005, 0.01, 0.05, 0.1\}$, evaluate top-$k \in \{5, 10, 20, 50, 100\}$, and report, for each metric, the best result across these settings, following standard practice in prior work.

These baselines do not scale to our largest schemas. In particular, they cannot process SISU, Restaurant, or the HMDA datasets with all columns present. For those datasets, we therefore restrict the input to the subset of columns that appear in the gold constraints, giving the profiling baselines the best possible setting under their scalability limits, and run discovery on a seeded sample of 100k of the 1M HMDA rows. On HMDA Checks (39 columns), no configuration finished: runs timed out (6 h with 100k rows, 2 h with 50k) or exhausted a 12 GB heap even with samples of 20k and 10k rows.

**Text-to-SQL.** We use XiYan-SQL [34], a state-of-the-art generative text-to-SQL framework. We adapt it to our setting by prompting it, for each sentence, to return a SQL query that identifies the tuples violating the described constraint, or `NULL` if the sentence does not express a checkable data constraint. Under this formulation, XiYan-SQL acts jointly as a constraint classifier and mapper, making it directly comparable to `Norma` across all evaluation dimensions. We use the 32B model with three seeds and report the mean.

## 6.3 Evaluation Metrics

Text-to-Constraints requires solving three related problems: identify text that expresses constraints, translate those into a formal representation, and identify the data violations implied by those constraints. We therefore evaluate systems along three complementary dimensions: *Constraint Classification*, *Constraint Mapping*, and *Violation Detection*. This is similar in spirit to Text-to-SQL evaluation, where execution-based evaluation complements structural matching because exact form alone does not fully capture correctness [62]. Because `Norma` and the baselines in Section 6.2 differ in what they produce, not all dimensions apply to every method.

**Constraint Classification.** This dimension measures whether a system correctly identifies which sentences in a document express constraints. We report precision, recall, and $F_1$ over all sentences. This dimension applies to sentence-level variants of `Norma` and to the Text-to-SQL baseline. It does not apply to data profiling baselines, which operate on data rather than text.

**Constraint Mapping.** This dimension measures how accurately systems translate text into denial constraints. We compare predicted and ground-truth constraints by predicate-set match (i.e., exact matching over the set of predicates), and report precision, recall, and $F_1$. For `Norma` and the data profiling baselines, the predicate set is read directly from the predicted denial constraint. For Text-to-SQL, we extract predicates from the `WHERE` clause of the generated query. A correlated `EXISTS` is read as a pair DC; a query with no DC reading, e.g., with aggregation or `CASE`, counts as a correct mapping if it returns exactly the violations of the gold DC of its sentence, so that equivalent SQL formulations are not penalized. For data profiling, discovered constraints are matched directly against the ground-truth set.

**Violation Detection.** This dimension measures whether the constraints produced by a system, when executed on the dataset, identify the same tuples as the ground-truth constraints. Because a denial constraint may involve either a single tuple or a pair of tuples, we evaluate violations at the tuple level by counting tuples that participate in at least one violation. We execute all predicted constraints and all ground-truth constraints on the dataset, then compare the resulting violation sets using precision, recall, and $F_1$. This dimension applies to all `Norma` variants and all baselines.

**Complementary mapping metrics.** Exact predicate-set matching penalizes logically equivalent rewritings, and the tuple-level aggregate can be dominated by a single rule. We therefore report four complementary measures for the mappers. (i) *Equivalence-aware mapping* ($CM_{\equiv}$) compares DCs after a logical normal form (e.g., `IN` as a disjunction, sorted operands, distributed `AND`/`OR`); in addition, literals on a code column and on its label column (e.g., `hoepa_status = 1` and `hoepa_status_name = 'High-cost mortgage'`) are unified only when the data confirms that they select the same tuples. (ii) *Execution equivalence* is the fraction of gold rules with at least one violation whose predicted DC flags exactly the same tuples. (iii) *Per-rule violation* $F_1$ averages, over gold sentences, the $F_1$ between the tuples flagged by the gold and the predicted DC, so that every rule weighs the same. (iv) *Violation concentration* is the share of the gold-violating tuples flagged by the single largest gold rule. We also count predicted DCs whose SQL fails to execute, instead of treating them as producing no violation.

## 6.4 Implementation and Execution Settings

`Norma` is implemented in Python, with DuckDB as the query execution backend. All LLM calls are issued through OpenRouter with strict JSON-schema structured outputs. We evaluate six backbones: GPT-6 Luna, DeepSeek-V4-Flash, Mistral Large 4, Qwen3.8-2.4T-A95B, GPT-6 Sol, and Claude Sonnet 5.5. Two candidates were replaced: Kimi K2.6, which repeatedly emitted whitespace until the token limit under strict JSON-schema decoding, and MiniMax-M3, which returned empty single-prompt outputs in the experiments (Section 6.6). Each configuration is executed once, with a cap of 32k output tokens per call. Unless stated otherwise, `Norma` uses the schema-driven predicate space of Section 5.1 with on-demand predicate generation, and the single-prompt mapper uses semantic column retrieval and column context (Section 5.2). Constraints are executed on the full datasets (first 1M rows for HMDA) with DuckDB. `Norma` experiments were executed on a Linux workstation with an Intel Core i7-7700HQ CPU and 16 GB of memory, as were the DC discovery baselines (Java 21). The XiYan-SQL baseline, which requires GPU inference, was executed on an NVIDIA A100 SXM GPU (80 GB VRAM). The baselines are scored by the same evaluator, data, and query limits as `Norma`.

## 6.5 Classifier Performance

Our first experiment evaluates how model and prompt design affect the accuracy of constraint classification. Starting from the prompt in Figure 2 (Section 4), we derive four variants by enabling or disabling *constraint examples* and *table context*: *No Context*, *Table Context*, *Examples*, and *Examples + Table Context*. We evaluate all four variants across six LLMs and all six datasets. Figure 6 summarizes the results. Because sentences missed at this stage cannot be recovered later by the mapper, we require a recall of at least 0.90 on every dataset when selecting a default configuration (red dashed line).

On Hospital, Tax, SISU, and HMDA Checks, almost every configuration reaches recall 1.00, so the differences lie in precision. HMDAZ and Restaurant behave differently: there, recall is the bottleneck. On HMDAZ, most configurations recover only 4 of the 9 constraint sentences (recall 0.44). The missed sentences are regulatory definitions (e.g., when a mortgage is high-cost) that read as scope statements, although each implies a checkable condition on the data. Only three configurations satisfy the recall requirement on all six datasets: *Qwen3.8-2.4T-A95B* with *Examples* (mean $F_1$ 0.69), *GPT-6 Luna* with *Examples* (0.69), and *Qwen3.8-2.4T-A95B* with *No Context* (0.68). The qualifying configurations are dominated by *Examples*, and we select *Qwen3.8-2.4T-A95B* with *Examples* as the default classifier. Configurations with higher mean $F_1$ exist (e.g., DeepSeek-V4-Flash with *Examples + Table Context*, 0.82), but they miss more than half of the HMDAZ constraints.

Table context has two opposite effects. It is the only signal that raises precision on HMDA Checks: without it, the models label (almost) every edit as a constraint, and $F_1$ equals that of the trivial all-positive classifier (0.74). The non-constraint edits are phrased like the checkable ones but refer to aggregates or external data, which the record layout in the table context presumably exposes. With table context, $F_1$ rises to 0.80–0.85. On wide schemas, however, the same context does not help recall: on HMDAZ (135 columns), 9 of the 12 table-context configurations stay at or below 0.44, and only GPT-6 Luna and Claude Sonnet 5.5 with *Examples + Table Context* recover all nine constraint sentences. Averaged over all models and datasets, *Examples* gives the best recall (0.93), while the table-context variants give higher precision (0.71 vs. 0.58–0.62) at the cost of recall (0.85–0.87).

Compared with earlier backbones that we evaluated with the same protocol in a preliminary study (e.g., Qwen3-32B and GPT-5), the current models are better on average on Hospital, Restaurant, SISU, and Tax (mean $F_1$ 0.72 vs. 0.69), but they accept more non-constraint sentences, so the best qualifying configuration is weaker (0.66 vs. 0.85 for Qwen3-32B with *Examples*). Without examples or table context, classification is already strong on the two controlled datasets (recall 1.00 and mean $F_1$ 0.72 on Tax and Hospital), and the best prompt variant adds only 0.04 on average. On SISU, Restaurant, and the two HMDA datasets, the gain is larger (mean $F_1$ from 0.66 to 0.75 with *Examples + Table Context*), mostly through precision; the GPT-6 models gain the least (0.03–0.05). A plausible explanation is that the LLM-written controlled sentences resemble text these models produce or saw in pretraining; we cannot verify such exposure, but the larger gains on real regulatory text suggest that context matters most there.

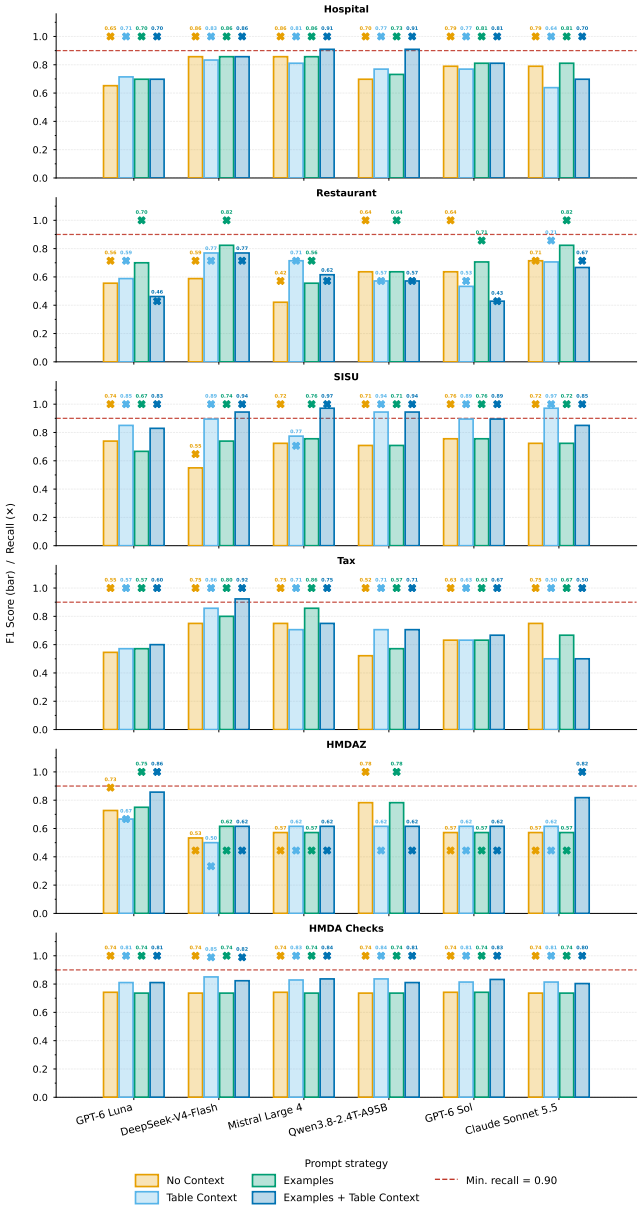



**Figure 6: Constraint classification $F_1$ (bars) and recall (x) across models and prompt variants. The dashed line marks the 0.90 recall threshold.**

## 6.6 Constraint Mapping Performance

Our second experiment evaluates how model and mapper choices affect mapping quality. We evaluate mapping quality using *constraint mapping* $F_1$ and *aggregate violation* $F_1$ as metrics. To isolate mapper quality from classification errors, both metrics are computed only on the ground-truth constraint sentences. We compare the *Single-Prompt* and *Multi-Step* mappers across six LLMs on all six datasets.

Figure 7 summarizes the results. On Hospital, Restaurant, SISU, and Tax, *Multi-Step* again improves both constraint mapping and aggregate violation recovery over *Single-Prompt* (mean over models: 0.81 vs. 0.75 and 0.83 vs. 0.55). The largest gains are on Restaurant and SISU, whose rules combine several conditions or relate two tuples. Hospital and Tax are easy for most configurations. With the current backbones, the best Multi-Step configurations exceed the best configuration of the preliminary study (GPT-5, 0.78 mapping and 0.95 violation): Claude Sonnet 5.5 reaches 0.85 and 0.97, GPT-6 Luna 0.82 and 0.98, and Qwen3.8-2.4T-A95B 0.80 and 1.00. Decomposition also keeps the mapper usable when a backbone is unstable under the single-prompt formulation: a backbone we initially included, MiniMax-M3, returned empty outputs for most Single-Prompt batches (107 of 156 sentences counted as output failures) but completed the Multi-Step pipeline without failures; we replaced it with Mistral Large 4, and among the six final backbones only DeepSeek-V4-Flash has output failures (3 Single-Prompt sentences).

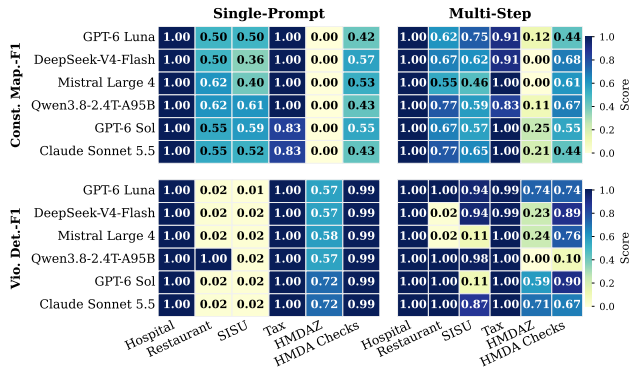



**Figure 7: Sentence-level mapping results across models and mappers, measured by constraint mapping $F_1$ and aggregate violation $F_1$ on ground-truth sentences only.**

**Where Single-Prompt loses violations.** The aggregate gap on Restaurant and SISU has two distinct causes. On Restaurant, one rule (at least one inspection per year) flags 99% of the gold-violating tuples. Every Single-Prompt configuration except Qwen3.8-2.4T-A95B returns `null` for it, so aggregate recall falls to 0.01. Once this rule is excluded, all of them reach 1.00. On SISU, the Single-Prompt mapper identifies the student by the exam registration number instead of the national ID in the two-tuple rules. The resulting DCs flag about 51k tuples instead of 533 (precision below 0.01). Multi-Step avoids both errors with most backbones because it links columns and selects predicates before writing the DC, but not with all of them: DeepSeek-V4-Flash and Mistral Large 4 also miss the Restaurant rule, and GPT-6 Sol and Mistral Large 4 make the same identification error on SISU in one rule (8.9k tuples; aggregate 0.11).

**HMDA datasets.** The two HMDA datasets remain the hardest. Exact mapping stays low (at most 0.25 on HMDAZ and 0.68 on HMDA Checks), mainly for two reasons. Many predicted DCs are logically equivalent to the gold ones but written differently, e.g., `IN` instead of a disjunction, or a label column instead of its code column. Others omit the coverage conditions of a trigger. Equivalence-aware mapping (Table 5) recovers a large part of the first group: on the HMDA datasets it raises the mean mapping $F_1$ from 0.34 to 0.46 for Multi-Step and from 0.24 to 0.51 for Single-Prompt, whereas on the other datasets it is almost identical to exact mapping. On HMDA Checks, aggregate violation is not informative. Two date-range edits flag 99% of the gold-violating tuples, which cover 98.6% of the rows, so a DC that flags every tuple already scores 0.99. Single-Prompt reaches 0.99 with every backbone, mostly through such DCs. Multi-Step scores 0.67–0.90 with five backbones, but only GPT-6 Sol maps one of the date edits correctly; the others write incorrect DCs for other edits that flag 480k–550k rows and pass the 90% violation-share check. Without the two date edits, these broad DCs are exposed: the Multi-Step aggregate falls to 0.12–0.19 for those four backbones and stays at 0.98–1.00 for GPT-6 Sol and Qwen3.8-2.4T-A95B, while Single-Prompt ranges from 0.18 to 1.00. Per-rule metrics, which weigh every rule equally, are far less sensitive: the mean per-rule violation $F_1$ is 0.78 for Multi-Step and 0.86 for Single-Prompt with all edits, and 0.84 and 0.85 without the date edits.

| Mapper | Model | CM | $CM_{\equiv}$ | AV | Rule-$F_1$ | Exec. | Fail. |
|---|---|---|---|---|---|---|---|
| Single-Prompt | GPT-6 Luna | 0.57 | 0.69 | 0.60 | 0.78 | 0.75 | 0 |
| | DeepSeek-V4-Flash | 0.57 | 0.66 | 0.60 | 0.74 | 0.70 | 3 |
| | Mistral Large 4 | 0.59 | 0.66 | 0.60 | 0.73 | 0.71 | 0 |
| | Qwen3.8-2.4T-A95B | 0.61 | 0.70 | 0.76 | **0.86** | **0.83** | 0 |
| | GPT-6 Sol | 0.59 | 0.67 | 0.62 | 0.76 | 0.72 | 0 |
| | Claude Sonnet 5.5 | 0.55 | 0.64 | 0.62 | 0.76 | 0.70 | 0 |
| Multi-Step | GPT-6 Luna | 0.64 | 0.69 | **0.90** | 0.77 | 0.76 | 0 |
| | DeepSeek-V4-Flash | 0.65 | 0.69 | 0.68 | 0.68 | 0.69 | 0 |
| | Mistral Large 4 | 0.60 | 0.64 | 0.52 | 0.74 | 0.73 | 0 |
| | Qwen3.8-2.4T-A95B | 0.66 | 0.68 | 0.68 | 0.81 | 0.77 | 0 |
| | GPT-6 Sol | 0.67 | 0.71 | 0.77 | 0.79 | 0.77 | 0 |
| | Claude Sonnet 5.5 | **0.68** | **0.74** | 0.88 | 0.84 | 0.82 | 0 |

**Table 5: Mean mapping quality over the six datasets (ground-truth sentences only). CM: exact mapping $F_1$; $CM_{\equiv}$: equivalence-aware mapping $F_1$; AV: aggregate violation $F_1$; Rule-$F_1$: mean per-rule violation $F_1$; Exec.: execution equivalence; Fail.: sentences with no valid model output.**

**Default mapper.** No single model leads on both metrics of the paper protocol: across the six datasets, Claude Sonnet 5.5 has the best mean mapping (0.68) and GPT-6 Luna the best mean aggregate violation (0.90) with Multi-Step. We use the mean of the two to break the tie and select *Claude Sonnet 5.5* with *Multi-Step* (mean 0.776, against 0.771 for GPT-6 Luna and 0.720 for GPT-6 Sol) as the default sentence-level configuration. With a single run per configuration, this is a tie with GPT-6 Luna.

**Sensitivity to the backbone.** Backbones differ more in violation detection than in mapping. With Multi-Step, mean exact mapping ranges only from 0.60 (Mistral Large 4) to 0.68 (Claude Sonnet 5.5), whereas aggregate violation ranges from 0.52 to 0.90; per-rule metrics lie in between (0.68–0.84) and rank Claude Sonnet 5.5, Qwen3.8-2.4T-A95B, and GPT-6 Sol highest. The divergences concentrate on implicit decisions rather than on syntax: which attribute identifies an entity (SISU), whether a definition implies a

checkable trigger (HMDAZ), and how integer-encoded dates are compared (HMDA Checks). Finally, single runs vary: in two runs of Mistral Large 4 on Restaurant that differ only in a check that does not affect the dominant rule, the aggregate went from 1.00 to 0.02, because that rule was mapped in one run and not in the other. We cannot relate these differences to model size or training data, which are not disclosed for most backbones.

## 6.7 End-to-End Evaluation and Comparison

We now evaluate the full Text-to-Constraints pipeline end-to-end and compare `Norma` against the baselines from Section 6.2. For `Norma`, we use the default components selected in the previous experiments: *Qwen3.8-2.4T-A95B* with *Examples* for constraint classification and *Claude Sonnet 5.5* with *Multi-Step* for mapping. We report two variants of `Norma`: the full pipeline with the classifier and the Multi-Step mapper (C+MS), and the Multi-Step mapper alone, which relies on its own rule filter (Multi-Step). As baselines, we include (i) the best result from DC discovery, following the procedure previously described, and (ii) an adaptation of the Text-to-SQL tool XiYan-SQL, as described before. Both baselines are run on all six datasets, with the same data, gold constraints, and evaluator as `Norma` (Section 6.2). Each end-to-end configuration is executed three times, and Table 6 reports the mean for `Norma`. Table 6 shows the results on all six datasets using the three metrics introduced in Section 6.3: constraint classification, constraint mapping, and violation detection.

`Norma` is the only approach that produces gold constraints on every dataset. Constraint mapping $F_1$ is 1.00 on Hospital and Tax, 0.67 on Restaurant, 0.65 on SISU, 0.44–0.45 on HMDA Checks, and 0.14–0.25 on HMDAZ (0.35 with equivalence-aware matching), against 0.00–0.80 for XiYan-SQL and zero for DC discovery. Classification is also strong, with $F_1$ between 0.79 and 0.91 for C+MS, above XiYan-SQL (0.55–0.86) on every dataset. The pipeline therefore extends to the two HMDA datasets, with up to 135 columns and 1M rows, against at most 56 columns and 128k rows in the other datasets.

The classifier mainly improves precision. Compared with *Multi-Step* alone, *C+MS* raises classification precision on Hospital (0.81 vs. 0.48), Restaurant (0.89 vs. 0.65), and Tax (0.71 vs. 0.33). On Tax, this decides the aggregate violation score. Multi-Step alone maps about 12 non-constraint sentences per run, which were written to resemble constraints, to DCs; one of them turns "major cities exist within fixed states" into `t1.city = t2.city AND t1.state != t2.state`, which flags 52k tuples, so the aggregate falls to 0.02. C+MS filters most of them and reaches 0.70 on average (0.55–1.00). On the two HMDA datasets, both variants make almost the same decisions; on HMDAZ, they nevertheless write different DCs for the same triggers (mean exact mapping 0.14 vs. 0.25, equivalence-aware 0.35 for both), and the aggregate varies from 0.02 to 0.75 across runs.

The violation results expose two further failure modes. First, a broad DC on a non-constraint sentence can dominate the aggregate. On SISU, both variants map sentences that define quota categories, which our ground truth labels as non-constraints, to checks between two quota attributes that flag up to 12k tuples; the aggregate stays at 0.05–0.06, close to XiYan-SQL (0.08), although the gold rules are recovered well (per-rule violation $F_1$ 0.91–0.95, execution equivalence 0.86–0.90). Second, missing a dominant rule: on Restaurant, the yearly-inspection rule flags 99% of the gold violations, and Multi-Step alone returns no DC for it in all three runs, C+MS in two of them. The rule filter judges this requirement, which concerns inspections across years, as not expressible, although the derived attribute encodes it. The aggregate is therefore 0.02 for Multi-Step and 0.35 on average for C+MS, below DC discovery (0.58) and XiYan-SQL (0.52), while mapping reaches 0.67. Both failures depend on the backbone and the run: with GPT-6 Sol as the mapper, the pipeline reached 1.00 on Restaurant and up to 0.98 on SISU but 0.15 on Tax. On HMDA Checks, neither the aggregate of 0.71 nor XiYan-SQL's 0.99 is evidence of quality: 98.6% of the rows violate some gold rule, so broad queries score high; `Norma` maps no date-range edit, and its score comes from an incorrect DC that flags 481k tuples. The per-rule view is more informative there: 83%

| Family | Method | Constr. Classification | | | Constr. Mapping | | | Viol. Detection | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | P | R | $F_1$ | P | R | $F_1$ | P | R | $F_1$ |
| **Hospital** | | | | | | | | | | |
| `Norma` | C+MS | 0.81 | 1.00 | **0.89** | 1.00 | 1.00 | **1.00** | 1.00 | 1.00 | **1.00** |
| | Multi-Step | 0.48 | 1.00 | 0.65 | 1.00 | 1.00 | **1.00** | 1.00 | 1.00 | **1.00** |
| Profiling | DC Discovery | n/a | n/a | n/a | 0.00 | 0.00 | 0.00 | 0.83 | 1.00 | 0.91 |
| Text2SQL | XiYan-SQL | 0.51 | 1.00 | 0.68 | 0.66 | 1.00 | 0.80 | 0.83 | 1.00 | 0.91 |
| **Restaurant** | | | | | | | | | | |
| `Norma` | C+MS | 0.89 | 0.71 | **0.79** | 0.80 | 0.57 | **0.67** | 1.00 | 0.34 | 0.35 |
| | Multi-Step | 0.65 | 0.71 | 0.68 | 0.80 | 0.57 | **0.67** | 0.87 | 0.01 | 0.02 |
| Profiling | DC Discovery | n/a | n/a | n/a | 0.00 | 0.00 | 0.00 | 0.41 | 1.00 | **0.58** |
| Text2SQL | XiYan-SQL | 0.71 | 0.71 | 0.71 | 0.50 | 0.43 | 0.46 | 0.44 | 0.64 | 0.52 |
| **SISU** | | | | | | | | | | |
| `Norma` | C+MS | 0.67 | 1.00 | **0.80** | 0.65 | 0.65 | **0.65** | 0.03 | 0.98 | 0.06 |
| | Multi-Step | 0.65 | 1.00 | 0.79 | 0.65 | 0.65 | **0.65** | 0.03 | 0.97 | 0.05 |
| Profiling | DC Discovery | n/a | n/a | n/a | 0.00 | 0.00 | 0.00 | 0.01 | 1.00 | 0.01 |
| Text2SQL | XiYan-SQL | 0.92 | 0.65 | 0.76 | 0.42 | 0.29 | 0.35 | 0.07 | 0.76 | **0.08** |
| **Tax** | | | | | | | | | | |
| `Norma` | C+MS | 0.71 | 1.00 | **0.82** | 1.00 | 1.00 | **1.00** | 0.59 | 1.00 | **0.70** |
| | Multi-Step | 0.33 | 1.00 | 0.50 | 1.00 | 1.00 | **1.00** | 0.01 | 1.00 | 0.02 |
| Profiling | DC Discovery | n/a | n/a | n/a | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 | 0.01 |
| Text2SQL | XiYan-SQL | 0.38 | 1.00 | 0.55 | 0.33 | 0.83 | 0.48 | 0.01 | 1.00 | 0.01 |
| **HMDAZ** | | | | | | | | | | |
| `Norma` | C+MS | 0.86 | 0.89 | 0.87 | 0.13 | 0.15 | 0.14 | 0.26 | 1.00 | 0.38 |
| | Multi-Step | 0.96 | 0.89 | **0.92** | 0.25 | 0.26 | **0.25** | 0.48 | 1.00 | **0.64** |
| Profiling | DC Discovery | n/a | n/a | n/a | 0.00 | 0.00 | 0.00 | 0.34 | 0.25 | 0.29 |
| Text2SQL | XiYan-SQL | 0.88 | 0.78 | 0.82 | 0.00 | 0.00 | 0.00 | 0.00 | 0.37 | 0.01 |
| **HMDA Checks** | | | | | | | | | | |
| `Norma` | C+MS | 0.90 | 0.93 | 0.91 | 0.45 | 0.43 | 0.44 | 0.99 | 0.56 | 0.71 |
| | Multi-Step | 0.90 | 0.93 | **0.92** | 0.45 | 0.44 | **0.45** | 0.99 | 0.56 | 0.71 |
| Profiling | DC Discovery | n/a | n/a | n/a | DNF | DNF | DNF | DNF | DNF | DNF |
| Text2SQL | XiYan-SQL | 0.76 | 1.00 | 0.86 | 0.34 | 0.37 | 0.36 | 0.99 | 1.00 | **0.99** |

**Table 6: End-to-end comparison of `Norma` modes, Classifier+Multi-Step (C+MS) and Multi-Step, and the baselines across six datasets (`Norma`: mean of three runs). Profiling denotes DC discovery from data alone, and Text2SQL denotes XiYan-SQL adapted to generate violation queries from regulatory sentences; DNF marks configurations that did not finish within the time and memory limits. n/a indicates that the metric is not applicable; bold marks the best $F_1$ for each metric and dataset.**

of the gold rules with violations are reproduced exactly, and the mean per-rule violation $F_1$ is 0.77–0.78.

The baselines are competitive only on narrower parts of the experiments. The profiling baseline can recover meaningful violation signals when the data distribution strongly reflects the target rules, but it does not recover the gold constraint set itself, with constraint-mapping $F_1$ equal to zero on all datasets, and it does not finish on HMDA Checks. XiYan-SQL is competitive on Hospital (mapping 0.80), but its behavior is much less reliable on the remaining datasets. On Tax and HMDAZ in particular, it returns queries pointing to overly broad violation sets, with near-zero violation precision.

Finally, the dataset characteristics matter when interpreting the aggregate violation results. In four of the six datasets, a single gold rule flags most of the gold-violating tuples (Section 6.6). Methods that return broad violation sets can still overlap strongly with the gold violations, as with the profiling baseline on Hospital and Restaurant. Conversely, a correct pipeline can be penalized by one broad false positive, as on Tax and SISU. We therefore recommend reading the aggregate together with the per-rule metrics of Section 6.3. Overall, `Norma` is ahead of both baselines in classification and mapping on all six datasets; in violation detection, it is ahead on Hospital, Tax, and HMDAZ and behind on Restaurant, SISU (by 0.02), and HMDA Checks. The violation metric remains sensitive to individual broad DCs.

**Usability of the output.** The returned set stays small and reviewable: C+MS returns 6–25 DCs per run on Hospital, Restaurant, SISU, and Tax, 9 on HMDAZ, and 95 on HMDA Checks, close to the number of gold constraints (6–92). Users can rank the DCs by the number of tuples they flag, read the source sentence of each one, and use the optional review step of the pipeline (Section 3); the broad DCs discussed above stand out precisely by their violation counts.

**Costs.** The full sentence-level `Norma` pipeline is the most expensive variant because it requires multiple model calls per sentence. All reported experiments together cost US$166 through OpenRouter (E1 US$5.7, E2 US$52.9, three end-to-end runs US$107.2). With Claude Sonnet 5.5, an end-to-end run costs between US$0.4 and US$2.9 on Hospital, Restaurant, SISU, and Tax and about US$9.7 on HMDA Checks (158 sentences, 1M rows). The classifier makes the pipeline up to six times cheaper (Tax), because rejected sentences never reach the Multi-Step mapper; on HMDA Checks, where most sentences are constraints, both variants cost the same. Low-cost backbones such as GPT-6 Luna and DeepSeek-V4-Flash cost a few cents per dataset. By contrast, data profiling baselines face combinatorial search over the schema, and for some datasets we had to restrict the columns considered.

## 7 RELATED WORK

**Data Profiling.** The closest line of work to `Norma` is *data profiling*, which aims to discover constraints directly from data instances [2, 9, 14, 20, 26, 42, 43, 46]. This includes formalisms such as functional dependencies, conditional dependencies, and denial constraints (DCs). DCs are particularly relevant in our setting because they subsume keys, FDs, and ordering constraints [14], making them expressive enough to capture many cross-tuple rules in regulatory text. Approximate discovery tolerates violations in dirty data [38, 44], user-specific criteria help experts rank discovered rules [19], and LLMs now propose candidate rules from schemas and samples [8, 22, 31]. Still, all these methods are driven by the observed data. They reveal what the data seems to satisfy, but not whether that pattern is required by policy or regulation. As a result, profiling cannot recover constraints that are already violated in the released data, and discovered DCs often include many spurious constraints [37]. Our goal is to recover constraints from normative text and then use them to validate the data.

**Data Cleaning.** Data cleaning systems use constraints, statistical signals, or both to detect and repair errors [1, 15, 36, 40, 47, 50], including rules with ML predicates (REEs) [6] and LLM-based detectors [41, 60]. These systems complement our setting. Constraint-based methods are interpretable and auditable, but they assume that the relevant rules are already available. Data-driven methods reduce the need for manual rule specification, but without an external source of truth, they can struggle to distinguish rare valid values from genuine errors [40]. Our work addresses this missing step of obtaining such rules from regulatory documents. We use DCs over a controlled, schema-derived predicate space as the output formalism, so every constraint is executable and auditable, and, as prior work has shown, DCs can be checked efficiently at scale [28, 35, 45].

**Regulatory NLP.** A separate line of work uses NLP to analyze regulations and compliance-related documents, for example, by extracting rights and obligations or by checking whether requirement statements comply with legal or policy text [10, 13, 17, 24, 29, 39]. LLMs now extract structured rules from regulations [23, 25, 27] and translate financial regulations into compliance code [32], although they still struggle to apply long sets of interdependent rules [56, 58]. These approaches remain text-centered or target a fixed application model. Our problem starts from similar documents, but we need to connect regulatory statements to a relational schema and translate them into constraints whose violations can be checked in the data.

**Text-to-SQL.** Text-to-SQL is also related, since it maps natural-language input to a SQL query over a database schema [4, 21, 30, 34, 48, 49, 51, 52, 54, 59]. The difference lies in how the generated logic is used, as Text-to-SQL produces a query that answers a user question on the current database instance, whereas `Norma` aims at declarative invariants that should hold for all valid instances of the data. In addition to mapping text to schema elements, as in Text-to-SQL, it must also identify scope conditions and define violation semantics. Closer in output, LLMs generate OCL constraints from specifications of UML models [3, 5], and Doc2DB systems verify extracted relations against integrity constraints [61]; in both cases, the target schema is designed together with the constraints, whereas `Norma` grounds regulatory text in an independently published dataset.

## 8 CONCLUSION

A constraint inferred from data describes what the data happens to satisfy, whereas a constraint contained in governing text describes what the data is required to satisfy. This distinction has received limited attention in database research, even though many published

datasets are accompanied by documents that define data quality criteria at some level.

In this paper, we show that the gap between data and normative text can be addressed automatically. Our results indicate that lightweight classification is sufficient to detect most checkable requirements, and that a structured multi-step pipeline grounded in both text and schema can translate them into executable constraints while avoiding hallucinated predicates and literals.

This enables a new form of validation: `Norma` can reveal compliance failures even when they do not manifest as discoverable patterns in the data. In such settings, the normative document becomes part of the data specification, and keeping that specification aligned with evolving text emerges as a concrete challenge for data management. Because each constraint comes from the text, it remains valid while the regulation is unchanged and can be re-executed, also incrementally [28], as the data evolves; only a change in the text requires a new mapping. Our datasets have at most 158 sentences, whereas legal documents often have hundreds, with nested clauses that must first be segmented and resolved; since cost grows linearly with the number of sentences, we expect the pipeline to scale to them, but this remains to be shown.

## REFERENCES


[1] Ziawasch Abedjan, Xu Chu, Dong Deng, Raul Castro Fernandez, Ihab F. Ilyas, Mourad Ouzzani, Paolo Papotti, Michael Stonebraker, and Nan Tang. 2016. Detecting data errors: where are we and what needs to be done? *PVLDB* 9, 12 (Aug. 2016), 993–1004. https://doi.org/10.14778/2994509.2994518

[2] Ziawasch Abedjan, Lukasz Golab, and Felix Naumann. 2015. Profiling relational data: a survey. *The VLDB Journal* 24, 4 (Aug. 2015), 557–581. https://doi.org/10.1007/s00778-015-0389-y

[3] Seif Abukhalaf, Mohammad Hamdaqa, and Foutse Khomh. 2023. On Codex Prompt Engineering for OCL Generation: An Empirical Study. In *Proceedings of the 20th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 148–157. https://doi.org/10.1109/MSR59073.2023.00033

[4] Tarfah Alrashed, Madhup Sukoon, David R. Karger, and Natasha Noy. 2026. Developing and Benchmarking Verification Algorithms to Improve Text-to-SQL Generation. *Proc. VLDB Endow.* 19, 11 (2026), 3732–3744. https://doi.org/10.14778/3836663.3836721

[5] Hamza Attarwala, Moataz Chouchen, Mohammad Hamdaqa, and Omar Alam. 2026. Why Do LLMs Fail at OCL Generation? A Graph Reasoning Perspective. In *Proceedings of the 20th International Symposium on Empirical Software Engineering and Measurement (ESEM) (LIPIcs)*, Vol. 394. https://doi.org/10.4230/LIPIcs.ESEM.2026.48

[6] Xianchun Bao, Zian Bao, Binbin Bie, QingSong Duan, Wenfei Fan, Hui Lei, Daji Li, Wei Lin, Peng Liu, Zhicong Lv, Mingliang Ouyang, Shuai Tang, Yaoshu Wang, Qiyuan Wei, Min Xie, Jing Zhang, Xin Zhang, Runxiao Zhao, and Shuping Zhou. 2024. Rock: Cleaning Data by Embedding ML in Logic Rules. In *Companion of the 2024 International Conference on Management of Data (SIGMOD-Companion)*. 106–119. https://doi.org/10.1145/3626246.3653372

[7] Carlo Batini, Cinzia Cappiello, Chiara Francalanci, and Andrea Maurino. 2009. Methodologies for data quality assessment and improvement. *ACM Comput. Surv.* 41, 3, Article 16 (July 2009), 52 pages. https://doi.org/10.1145/1541880.1541883

[8] Fabian Biester, Mohamed Abdelaal, and Daniel Del Gaudio. 2024. LLM-Clean: Context-Aware Tabular Data Cleaning via LLM-Generated OFDs. *CoRR* abs/2404.18681 (2024).

[9] Tobias Bleifuß, Sebastian Kruse, and Felix Naumann. 2017. Efficient Denial Constraint Discovery with Hydra. *PVLDB* 11, 3 (2017), 311–323.

[10] Travis D. Breaux, Matthew W. Vail, and Annie I. Antón. 2006. Towards Regulatory Compliance: Extracting Rights and Obligations to Align Requirements with Regulations. In *Proceedings of the 14th IEEE International Requirements Engineering Conference (RE)*. IEEE Computer Society, 46–55.

[11] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) *(NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 159, 25 pages.

[12] Ismael Caballero, Fernando Gualo, Moisés Rodríguez, and Mario Piattini. 2022. BR4DQ: A methodology for grouping business rules for data quality evaluation. *Information Systems* 109 (2022), 102058. https://doi.org/10.1016/j.is.2022.102058

[13] Orlando Amaral Cejas, Muhammad Ilyas Azeem, Sallam Abualhaija, and Lionel C. Briand. 2023. NLP-Based Automated Compliance Checking of Data Processing Agreements Against GDPR. *IEEE Trans. Softw. Eng.* 49, 9 (Sept. 2023), 4282–4303. https://doi.org/10.1109/TSE.2023.3288901

[14] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Discovering denial constraints. *PVLDB* 6, 13 (Aug. 2013), 1498–1509. https://doi.org/10.14778/2536258.2536262

[15] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Holistic data cleaning: Putting violations into context. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*. 458–469. https://doi.org/10.1109/ICDE.2013.6544847

[16] Kevin L. Cope. 2023. Measuring law's normative force. *Journal of Empirical Legal Studies* 20, 4 (2023), 1005–1044. https://doi.org/10.1111/jels.12364 arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/jels.12364

[17] Souvick Das, Novarun Deb, Nabendu Chaki, and Agostino Cortesi. 2025. A Multi-Agent RAG Framework for Regulatory Compliance Checking of Software Requirements. *ACM Trans. Softw. Eng. Methodol.* (Dec. 2025). https://doi.org/10.1145/3785472 Just Accepted.

[18] Ruanitto Docini, Eduardo C. de Almeida, and Luiz S. Oliveira. 2025. The Wrecking SQL Incremental Validation Methodology. In *Database and Expert Systems Applications - 36th International Conference, DEXA 2025, Bangkok, Thailand, August 25-27, 2025, Proceedings, Part II (Lecture Notes in Computer Science)*. Springer, 376–382. https://doi.org/10.1007/978-3-032-02088-8_30

[19] Wenfei Fan, Ziyan Han, Yaoshu Wang, and Min Xie. 2023. Discovering Top-k Rules using Subjective and Objective Criteria. *Proc. ACM Manag. Data* 1, 1 (2023), 1–29. https://doi.org/10.1145/3588924

[20] Wenfei Fan, Jianhua Li, Shuai Ma, and Nan Tang. 2009. Discovering Conditional Functional Dependencies. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 1231–1234.

[21] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.* 17, 5 (Jan. 2024), 1132–1145. https://doi.org/10.14778/3641204.3641221

[22] Anna-Christina Glock, Thomas Hütter, Johannes Fürnkranz, Wolfram Wöß, Christine Dominka-Kiss, and Lisa Ehrlinger. 2026. Data Quality Rule Generation with LLMs. *CoRR* abs/2609.06053 (2026).

[23] Keerat Guliani, Deepkamal Gill, David Landsman, Nima Eshraghi, Krishna Kumar, and Lovedeep Gondara. 2026. De Jure: Iterative LLM Self-Refinement for Structured Extraction of Regulatory Rules. *CoRR* abs/2604.02276 (2026).

[24] Shabnam Hassani, Mehrdad Sabetzadeh, Daniel Amyot, and Jain Liao. 2024. Rethinking Legal Compliance Automation: Opportunities with Large Language Models. In *Proceedings of the 32nd IEEE International Requirements Engineering Conference (RE)*. 432–440. https://doi.org/10.1109/RE59067.2024.00051

[25] Hansi Hettiarachchi, Amna Dridi, Mohamed Medhat Gaber, Pouyan Parsafard, Nicoleta Bocaneala, Katja Breitenfelder, Gonçal Costa, Maria Hedblom, Mihaela Juganaru-Mathieu, Thamer Mecharnia, Sumee Park, He Tan, Abdel-Rahman H. Tawil, and Edlira Vakaj. 2025. CODE-ACCORD: A Corpus of building regulatory data for rule generation towards automatic compliance checking. *Scientific Data* 12, 1 (2025), 170.

[26] Ykä Huhtala, Juha Kärkkäinen, Pasi Porkka, and Hannu Toivonen. 1998. TANE: An Efficient Algorithm for Discovering Functional and Approximate Dependencies. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 392–401.

[27] Zarina Kabzhan, Alexandr Shakhnovich, Sergey Gorshkov, Yussuf Yemenov, Fedor Gorshkov, and Nazym Shogelova. 2025. Semantic and ontology-based analysis of regulatory documents for construction industry digitalization. *Frontiers in Built Environment* 11 (2025), 1575913. https://doi.org/10.3389/fbuil.2025.1575913

[28] Youri Kaminsky, Eduardo H. M. Pena, and Felix Naumann. 2025. Incremental Detection of Denial Constraint Violations. *PVLDB* 18, 4 (May 2025), 1000–1012. https://doi.org/10.14778/3717755.3717761

[29] Nadzeya Kiyavitskaya, Nicola Zeni, Travis D. Breaux, Annie I. Antón, James R. Cordy, Luisa Mich, and John Mylopoulos. 2008. Automating the Extraction of Rights and Obligations for Regulatory Compliance. In *Conceptual Modeling - ER 2008, 27th International Conference on Conceptual Modeling, Barcelona, Spain, October 20-24, 2008. Proceedings (Lecture Notes in Computer Science)*, Vol. 5231. Springer, 154–168.

[30] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. RESDSQL: decoupling schema linking and skeleton parsing for text-to-SQL. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'23/IAAI'23/EAAI'23)*. AAAI Press, Article 1466, 9 pages. https://doi.org/10.1609/aaai.v37i11.26535

[31] Qing Li, Yanyan Shen, Qibin Zheng, Yi Liu, and Xingchun Diao. 2026. LCATS: LLM-Guided Constraint-Aware Tabular Data Synthesis. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*

*(KDD)*. 2698–2707. https://doi.org/10.1145/3770855.3818181

[32] Siyuan Li, Jian Chen, Rui Yao, Xuming Hu, Peilin Zhou, Weihua Qiu, Simin Zhang, Chucheng Dong, Zhiyao Li, Qipeng Xie, and Zixuan Yuan. 2025. Compliance-to-Code: Enhancing Financial Compliance Checking via Code Generation. *CoRR* abs/2505.19804 (2025).

[33] Xi Victoria Lin, Richard Socher, and Caiming Xiong. 2020. Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 4870–4888. https://doi.org/10.18653/v1/2020.findings-emnlp.438

[34] Yifu Liu, Yin Zhu, Yingqi Gao, Zhiling Luo, Xiaoxia Li, Xiaorong Shi, Yuntao Hong, Jinyang Gao, Yu Li, Bolin Ding, and Jingren Zhou. 2026. XiYan-SQL: A Novel Multi-Generator Framework for Text-to-SQL. *IEEE Transactions on Knowledge and Data Engineering* (2026), 1–14. https://doi.org/10.1109/TKDE.2026.3657851

[35] Zifan Liu, Shaleen Deep, Anna Fariha, Fotis Psallidas, Ashish Tiwari, and Avrilia Floratou. 2024. Rapidash: Efficient Detection of Constraint Violations. *Proc. VLDB Endow.* 17, 8 (April 2024), 2009–2021. https://doi.org/10.14778/3659437.3659454

[36] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 865–882. https://doi.org/10.1145/3299869.3324956

[37] Albert Martin, Eduardo C. de Almeida, Oscar Romero, and Anna Queralt. 2025. How and Why False Denial Constraints are Discovered. *PVLDB* 18, 10 (2025), 3477–3489.

[38] Albert Martin, Eduardo C. de Almeida, Oscar Romero, and Anna Queralt. 2026. Discovering Approximate Denial Constraints in Large Databases. *Proc. VLDB Endow.* 19, 9 (2026), 2086–2098. https://doi.org/10.14778/3819518.3819536

[39] Arsalan Masoudifard, Mohammad Mowlavi Sorond, Moein Madadi, Mohammad Sabokrou, and Elahe Habibi. 2024. Leveraging Graph-RAG and Prompt Engineering to Enhance LLM-Based Automated Requirement Traceability and Compliance Checks. *CoRR* abs/2412.08593 (2024).

[40] Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, and Jianwei Yin. 2024. Automatic Data Repair: Are We Ready to Deploy? *PVLDB* 17, 10 (June 2024), 2617–2630. https://doi.org/10.14778/3675034.3675051

[41] Wei Ni, Kaihang Zhang, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Yaoshu Wang, and Jianwei Yin. 2025. ZeroED: Hybrid Zero-Shot Error Detection Through Large Language Model Reasoning. In *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE)*. 3126–3139. https://doi.org/10.1109/ICDE65448.2025.00234

[42] Thorsten Papenbrock and Felix Naumann. 2016. A Hybrid Approach to Functional Dependency Discovery. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 821–833.

[43] Eduardo Pena, Fabio Porto, and Felix Naumann. 2024. Discovering Denial Constraints in Dynamic Datasets. 3546–3558. https://doi.org/10.1109/ICDE60146.2024.00273

[44] Eduardo H. M. Pena, Eduardo C. de Almeida, and Felix Naumann. 2019. Discovery of Approximate (and Exact) Denial Constraints. *PVLDB* 13, 3 (2019), 266–278.

[45] Eduardo H. M. Pena, Eduardo C. de Almeida, and Felix Naumann. 2021. Fast detection of denial constraint violations. *PVLDB* 15, 4 (Dec. 2021), 859–871. https://doi.org/10.14778/3503585.3503595

[46] Eduardo H. M. Pena, Fabio Porto, and Felix Naumann. 2022. Fast Algorithms for Denial Constraint Discovery. *PVLDB* 16, 4 (2022), 684–696.

[47] Jinfeng Peng, Derong Shen, Nan Tang, Tieying Liu, Yue Kou, Tiezheng Nie, Hang Cui, and Ge Yu. 2022. Self-Supervised and Interpretable Data Cleaning with Sequence Generative Adversarial Networks. *Proc. VLDB Endow.* 16, 3 (Nov. 2022), 433–446. https://doi.org/10.14778/3570690.3570694

[48] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: decomposed in-context learning of text-to-SQL with self-correction. In *Proceedings of the 37th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA, Article 1577, 10 pages.

[49] Ricardo Trainotti Rabonato and Lilian Berton. 2026. The Impact of Language and Portuguese Intralinguistic Variation on NL2SQL. In *Anais do XVII Simpósio Brasileiro de Tecnologia da Informação e da Linguagem Humana (STIL)*. 317–330. https://doi.org/10.5753/stil.2026.26611

[50] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: holistic data repairs with probabilistic inference. *PVLDB* 10, 11 (Aug. 2017), 1190–1201. https://doi.org/10.14778/3137628.3137631

[51] Tonghui Ren, Chen Ke, Yuankai Fan, Yinan Jing, Zhenying He, Kai Zhang, and X. Sean Wang. 2025. The Power of Constraints in Natural Language to SQL Translation. *Proc. VLDB Endow.* 18, 7 (March 2025), 2097–2111. https://doi.org/10.14778/3734839.3734847

[52] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 9895–9901. https://doi.org/10.18653/v1/2021.emnlp-main.779

[53] Flavia Serra, Verónika Peralta, Adriana Marotta, and Patrick Marcel. 2024. Use of Context in Data Quality Management: A Systematic Literature Review. *J. Data and Information Quality* 16, 3, Article 19 (Oct. 2024), 41 pages. https://doi.org/10.1145/3672082

[54] Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang. 2025. A Survey on Employing Large Language Models for Text-to-SQL Tasks. *ACM Comput. Surv.* 58, 2, Article 54 (Sept. 2025), 37 pages. https://doi.org/10.1145/3737873

[55] Shaoxu Song, Fei Gao, Ruihong Huang, and Chaokun Wang. 2020. Data Dependencies over Big Data: A Family Tree. *IEEE Transactions on Knowledge and Data Engineering* PP (12 2020), 1–1. https://doi.org/10.1109/TKDE.2020.3046443

[56] Utkarsh Soni, Syed Shariyar Murtaza, Yifan Nie, Sachin Chandrasekhar, and Eugene Wen. 2026. Tasks over Application Manuals: Revealing Gaps in Long-Horizon Procedural Reasoning for Language Models. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM)*. https://doi.org/10.1145/3799682.3841039

[57] Xiaofei Sun, Xiaoya Li, Jiwei Li, Fei Wu, Shangwei Guo, Tianwei Zhang, and Guoyin Wang. 2023. Text Classification via Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 8990–9005. https://doi.org/10.18653/v1/2023.findings-emnlp.603

[58] Bohan Yu, Pengfei Cao, Chen Han, Chenxi Zhou, Zhiheng Zhang, Zhiyang Xie, Wenhao Teng, Xiangwen Liao, Jun Zhao, and Kang Liu. 2026. Beyond Factual Knowledge: Benchmarking and Learning Step-Level Procedural Rule Reasoning in Large Language Models. *CoRR* abs/2608.22753 (2026).

[59] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Brussels, Belgium, 3911–3921.

[60] Shuo Zhang, Zezhou Huang, and Eugene Wu. 2024. Data Cleaning Using Large Language Models. *CoRR* abs/2410.15547 (2024).

[61] Zhengxuan Zhang, Zhuowen Liang, Jiazhuo Chen, Haixun Wang, and Nan Tang. 2026. Document-to-Database: Extraction Meets Relational Semantics. *Proc. VLDB Endow.* 19, 9 (2026), 2522–2532. https://doi.org/10.14778/3819518.3819568

[62] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Bonnie Webber, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 396–411. https://doi.org/10.18653/v1/2020.emnlp-main.29